

Q-Commerce Supply Chain & Operations Analytics using Python

Project Overview

An end-to-end Python analytics project focused on understanding and optimizing the operations of a quick-commerce business. Using a synthetic Swiggy Instamart-inspired dataset containing orders, customers, products, stores, inventory, delivery partners, payments, and promotional data, the project analyzes revenue, customer value, inventory, demand, delivery operations, and fleet performance.

The project begins with comprehensive data cleaning and validation, followed by exploratory data analysis, predictive modeling, forecasting, and optimization techniques to generate actionable business insights.

Key Analysis & Modeling Areas

Revenue Trends — Analyze revenue patterns across time, cities, stores, products, and categories.

Fleet Utilization — Evaluate delivery-partner workload, vehicle utilization, order volume, and delivery efficiency.

CLV Prediction — Identify and predict high-value customers based on purchasing behavior and customer characteristics.

Stock-Out Prediction — Identify products and stores at higher risk of stockouts using historical inventory and demand patterns.

Demand Forecasting — Forecast future product/store demand to support operational and inventory planning.

Route Optimization — Analyze delivery operations and identify opportunities to improve delivery routes and partner allocation.

Inventory Replenishment Strategies — Develop data-driven replenishment recommendations based on demand, inventory levels, and stockout risk.

Dark Store Capacity Planning — Evaluate store capacity, order volume, and operational pressure to identify potential capacity constraints.

Fleet Mix Optimization — Analyze the performance of different vehicle types and determine an efficient fleet allocation strategy.

Python Skills & Technologies

Python | Pandas | NumPy | Matplotlib | Seaborn | SciPy | Scikit-learn | Time Series Forecasting | Machine Learning | Optimization | Exploratory Data Analysis

Business Objective

The primary objective is to use Python-based analytics and predictive techniques to identify operational inefficiencies, anticipate demand and inventory risks, improve delivery operations, and support data-driven supply-chain decisions in a quick-commerce environment.

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.ensemble import (
    RandomForestRegressor,
    RandomForestClassifier,
    GradientBoostingRegressor,
    GradientBoostingClassifier
)
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix
)
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.statespace.sarimax import SARIMAX
from scipy.optimize import minimize, linprog
from sklearn.cluster import KMeans
from datetime import datetime, timedelta
pd.set_option("display.max_columns", None)
pd.set_option("display.float_format", "{:.2f}".format)
sns.set_theme(style="whitegrid")
```

```
In [2]: df1 = pd.read_csv(r"C:\Users\papus\OneDrive\Documents\Swiggy Instamart Project\01_products.csv")
df1.head()
```

Out[2]:

	ProductID	ProductName	Category	SubCategory	Brand	MRP	SellingPrice	DiscountPercent
0	PROD00001	Brooke Bond Tea 1pc	Beverages	Tea	Brooke Bond	554.17	480.45	13.30
1	PROD00002	Purepet Pet Food 1kg	Pet Care	Pet Food	Purepet	256.88	214.71	16.42
2	PROD00003	Gerber Fresh Baby Wipes 5pcs	Baby Care	Baby Wipes	Gerber	130.41	109.41	16.10
3	PROD00004	India Gate Organic Spices 200ml	Atta & Grains	Spices	India Gate	38.08	31.51	17.26
4	PROD00005	Dhara Cooking Oil 10pcs	Oil & Masalas	Cooking Oil	Dhara	531.33	465.03	12.48

In [4]:

df1.shape

Out[4]: (1000, 16)

In [5]:

df1.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 16 columns):
Column Non-Null Count Dtype
--- ---
0 ProductID 1000 non-null object
1 ProductName 1000 non-null object
2 Category 1000 non-null object
3 SubCategory 1000 non-null object
4 Brand 1000 non-null object
5 MRP 1000 non-null float64
6 SellingPrice 1000 non-null float64
7 DiscountPercent 1000 non-null float64
8 Unit 1000 non-null object
9 InStock 1000 non-null object
10 StockQuantity 1000 non-null int64
11 ExpiryDate 259 non-null object
12 Rating 1000 non-null float64
13 ReviewCount 1000 non-null int64
14 IsVeg 1000 non-null int64
15 AddedDate 1000 non-null object
dtypes: float64(4), int64(3), object(9)
memory usage: 125.1+ KB

In [7]:

df1["ExpiryDate"] = pd.to_datetime(
 df1["ExpiryDate"],
 format="%d-%m-%Y",
 errors="coerce"
)

df1["AddedDate"] = pd.to_datetime(
 df1["AddedDate"],
 format="%d-%m-%Y",
 errors="coerce"
)

```
In [9]: df1[["ExpiryDate", "AddedDate"]].dtypes
```

```
Out[9]: ExpiryDate    datetime64[ns]  
AddedDate    datetime64[ns]  
dtype: object
```

```
In [10]: df1.columns.tolist()
```

```
Out[10]: ['ProductID',  
          'ProductName',  
          'Category',  
          'SubCategory',  
          'Brand',  
          'MRP',  
          'SellingPrice',  
          'DiscountPercent',  
          'Unit',  
          'InStock',  
          'StockQuantity',  
          'ExpiryDate',  
          'Rating',  
          'ReviewCount',  
          'IsVeg',  
          'AddedDate']
```

```
In [11]: df1.isnull().sum()
```

```
Out[11]: ProductID      0  
ProductName    0  
Category      0  
SubCategory    0  
Brand          0  
MRP            0  
SellingPrice   0  
DiscountPercent 0  
Unit           0  
InStock        0  
StockQuantity  0  
ExpiryDate     741  
Rating         0  
ReviewCount    0  
IsVeg          0  
AddedDate      0  
dtype: int64
```

```
In [12]: df1.duplicated().sum()
```

```
Out[12]: np.int64(0)
```

```
In [13]: # Check missing values in ExpiryDate  
missing_expiry = df1["ExpiryDate"].isna().sum()  
  
print("Missing ExpiryDate values:", missing_expiry)  
  
# Percentage missing  
missing_expiry_pct = df1["ExpiryDate"].isna().mean() * 100  
  
print(f"Missing ExpiryDate percentage: {missing_expiry_pct:.2f}%")
```

Missing ExpiryDate values: 741

Missing ExpiryDate percentage: 74.10%

```
In [14]: df1[df1["ExpiryDate"].isna()][
        ["ProductName", "Category", "SubCategory", "Unit", "IsVeg"]
        ].head(20)
```

Out[14]:

	ProductName	Category	SubCategory	Unit	IsVeg
0	Brooke Bond Tea 1pc	Beverages	Tea	2kg	1
1	Purepet Pet Food 1kg	Pet Care	Pet Food	200g	1
2	Gerber Fresh Baby Wipes 5pcs	Baby Care	Baby Wipes	250g	1
3	India Gate Organic Spices 200ml	Atta & Grains	Spices	10pcs	0
4	Dhara Cooking Oil 10pcs	Oil & Masalas	Cooking Oil	100g	1
7	Dhara Organic Pickles 10pcs	Oil & Masalas	Pickles	500g	1
8	Haldiram Premium Dry Fruits 250g	Snacks & Munchies	Dry Fruits	12pcs	1
9	Parle Organic Namkeen 250ml	Snacks & Munchies	Namkeen	100ml	0
10	Fortune Fresh Ghee 200g	Oil & Masalas	Ghee	200g	1
12	Parle Premium Namkeen 200ml	Snacks & Munchies	Namkeen	5kg	1
13	Huggies Baby Oil 500ml	Baby Care	Baby Oil	200ml	0
15	Patanjali Fresh Shampoo 1L	Personal Care	Shampoo	1L	1
16	India Gate Premium Pulses 200ml	Atta & Grains	Pulses	500g	1
17	Real Coffee 500ml	Beverages	Coffee	250ml	1
18	Coca Cola Organic Health Drinks 100ml	Beverages	Health Drinks	12pcs	1
19	Pepsi Organic Juices 250g	Beverages	Juices	10pcs	1
20	Bikaji Organic Dry Fruits 200ml	Snacks & Munchies	Dry Fruits	2kg	1
21	MTR Fresh Rice 250ml	Atta & Grains	Rice	6pcs	1
23	Gerber Baby Wipes 12pcs	Baby Care	Baby Wipes	200g	1
24	Purepet Pet Accessories 250g	Pet Care	Pet Accessories	250ml	1

```
In [16]: df1.groupby("Category")["ExpiryDate"].apply(
        lambda x: x.isna().sum()
        ).sort_values(ascending=False)
```

Out[16]:

Category	
Atta & Grains	105
Beverages	102
Oil & Masalas	99
Household	95
Personal Care	89
Snacks & Munchies	88
Pet Care	86
Baby Care	77
Bakery & Bread	0
Fruits & Vegetables	0
Dairy & Eggs	0
Name: ExpiryDate, dtype: int64	

```
In [17]: df2 = pd.read_csv(r"C:\Users\papus\OneDrive\Documents\Swiggy Instamart Project\02_stores.csv")
df2.head()
```

Out[17]:

	StoreID	StoreName	City	Zone	Pincode	StoreType	OpeningDate	Operatin
0	STORE001	Instamart Rohini Hub	Delhi NCR	Rohini	43321	Dark Store - Standard	04-06-2026	06:00 AM
1	STORE002	Instamart Guntur Hub	Andhra Pradesh	Guntur	133890	Dark Store - Standard	06-12-2025	06:00 AM
2	STORE003	Instamart Koramangala Hub	Bangalore	Koramangala	940265	Dark Store - Mega Hub	22-09-2024	06:00 AM
3	STORE004	Instamart Secunderabad Hub	Hyderabad	Secunderabad	161559	Dark Store - Mega Hub	01-09-2024	06:00 AM
4	STORE005	Instamart Vizag (Visakhapatnam) Hub	Andhra Pradesh	Vizag (Visakhapatnam)	618495	Dark Store - Standard	31-12-2025	06:00 AM

In [18]:

df2.shape

Out[18]: (50, 14)

In [21]:

df2.columns.tolist()

Out[21]: ['StoreID',
'StoreName',
'City',
'Zone',
'Pincode',
'StoreType',
'OpeningDate',
'OperatingHours',
'ManagerName',
'ContactNumber',
'Latitude',
'Longitude',
'Status',
'DailyOrderCapacity']

In [19]:

df2.info()

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   StoreID                50 non-null    object
1   StoreName              50 non-null    object
2   City                   50 non-null    object
3   Zone                   50 non-null    object
4   Pincode                50 non-null    int64
5   StoreType              50 non-null    object
6   OpeningDate            50 non-null    object
7   OperatingHours         50 non-null    object
8   ManagerName            50 non-null    object
9   ContactNumber          50 non-null    int64
10  Latitude               50 non-null    float64
11  Longitude              50 non-null    float64
12  Status                 50 non-null    object
13  DailyOrderCapacity     50 non-null    int64
dtypes: float64(2), int64(3), object(9)
memory usage: 5.6+ KB

```

```

In [22]: df2["OpeningDate"] = pd.to_datetime(
        df2["OpeningDate"],
        format="%d-%m-%Y",
        errors="coerce"
    )

```

```

In [23]: df2.dtypes

```

```

Out[23]: StoreID                object
StoreName              object
City                   object
Zone                   object
Pincode                int64
StoreType              object
OpeningDate            datetime64[ns]
OperatingHours         object
ManagerName            object
ContactNumber          int64
Latitude               float64
Longitude              float64
Status                 object
DailyOrderCapacity     int64
dtype: object

```

```

In [24]: df2.isnull().sum()

```

```

Out[24]: StoreID                0
StoreName              0
City                   0
Zone                   0
Pincode                0
StoreType              0
OpeningDate            0
OperatingHours         0
ManagerName            0
ContactNumber          0
Latitude               0
Longitude              0
Status                 0
DailyOrderCapacity     0
dtype: int64

```

```
In [25]: df2.duplicated().sum()
```

```
Out[25]: np.int64(0)
```

```
In [26]: df3 = pd.read_csv(r"C:\Users\papus\OneDrive\Documents\Swiggy Instamart Project\03_customers.csv")
df3.head()
```

Out[26]:

	CustomerID	FirstName	LastName	Email	PhoneNumber	City	Pincode
0	CUST00001	Isaac	Bakshi	isaac.bakshi60@email.com	9944499530	Mumbai	332181
1	CUST00002	Azad	Balan	azad.balan42@email.com	9772666902	Delhi NCR	338908
2	CUST00003	Oni	Kannan	oni.kannan29@email.com	9443072390	Bangalore	940265
3	CUST00004	Tanveer	Nayar	tanveer.nayar36@email.com	9451517302	Bangalore	161559
4	CUST00005	Caleb	Sharaf	caleb.sharaf55@email.com	9614117732	Andhra Pradesh	618495

```
In [27]: df3.shape
```

```
Out[27]: (5000, 14)
```

```
In [28]: df3.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   CustomerID            5000 non-null   object
1   FirstName              5000 non-null   object
2   LastName               5000 non-null   object
3   Email                  5000 non-null   object
4   PhoneNumber            5000 non-null   int64
5   City                   5000 non-null   object
6   Pincode                5000 non-null   int64
7   DateOfBirth           5000 non-null   object
8   Gender                 5000 non-null   object
9   RegistrationDate       5000 non-null   object
10  LoyaltyTier            5000 non-null   object
11  TotalLifetimeValue     5000 non-null   float64
12  TotalOrdersPlaced      5000 non-null   int64
13  IsActive               5000 non-null   int64
dtypes: float64(1), int64(4), object(9)
memory usage: 547.0+ KB
```

```
In [29]: df3["DateOfBirth"] = pd.to_datetime(
df3["DateOfBirth"],
format="%d-%m-%Y",
errors="coerce"
)

df3["RegistrationDate"] = pd.to_datetime(
df3["RegistrationDate"],
format="%d-%m-%Y",
errors="coerce"
)
```

```
In [31]: df3.dtypes
```

```
Out[31]: CustomerID      object
          FirstName      object
          LastName      object
          Email          object
          PhoneNumber    int64
          City           object
          Pincode        int64
          DateOfBirth    datetime64[ns]
          Gender         object
          RegistrationDate datetime64[ns]
          LoyaltyTier     object
          TotalLifetimeValue float64
          TotalOrdersPlaced int64
          IsActive       int64
          dtype: object
```

```
In [32]: df3.isnull().sum()
```

```
Out[32]: CustomerID      0
          FirstName      0
          LastName      0
          Email          0
          PhoneNumber    0
          City           0
          Pincode        0
          DateOfBirth    0
          Gender         0
          RegistrationDate 0
          LoyaltyTier     0
          TotalLifetimeValue 0
          TotalOrdersPlaced 0
          IsActive       0
          dtype: int64
```

```
In [34]: df3.duplicated().sum()
```

```
Out[34]: np.int64(0)
```

```
In [35]: df4 = pd.read_csv(r"C:\Users\papus\OneDrive\Documents\Swiggy Instamart Project\04_delivery_pa
df4.head()
```

Out[35]:

	PartnerID	FirstName	LastName	PhoneNumber	City	AssignedStoreID	VehicleType	DrivingL
0	PART0001	Daksh	Bakshi	9300753877	Bangalore	STORE049	Bike	AP35
1	PART0002	Laksh	Konda	9394319183	Delhi NCR	STORE045	Bicycle	AP42
2	PART0003	Ekaraj	Chaudry	9725068643	Bangalore	STORE018	Bike	AP76
3	PART0004	Caleb	Sami	9764441435	Delhi NCR	STORE031	Bike	AP47
4	PART0005	Alexander	Chander	9703976329	Bangalore	STORE023	Bike	AP33

```
In [36]: df4.shape
```

```
Out[36]: (800, 13)
```

```
In [37]: df4.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 800 entries, 0 to 799
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   PartnerID             800 non-null   object
1   FirstName              800 non-null   object
2   LastName               800 non-null   object
3   PhoneNumber            800 non-null   int64
4   City                  800 non-null   object
5   AssignedStoreID        800 non-null   object
6   VehicleType            800 non-null   object
7   DrivingLicenseNo       800 non-null   object
8   JoiningDate            800 non-null   object
9   TotalDeliveries        800 non-null   int64
10  PartnerRating           800 non-null   float64
11  AvgDeliveryTime_Min     800 non-null   float64
12  Status                 800 non-null   object
dtypes: float64(2), int64(2), object(9)
memory usage: 81.4+ KB
```

```
In [38]: df4["JoiningDate"] = pd.to_datetime(
        df4["JoiningDate"],
        format="%d-%m-%Y",
        errors="coerce"
    )
```

```
In [39]: df4.dtypes
```

```
Out[39]: PartnerID             object
FirstName             object
LastName              object
PhoneNumber            int64
City                  object
AssignedStoreID        object
VehicleType            object
DrivingLicenseNo       object
JoiningDate            datetime64[ns]
TotalDeliveries        int64
PartnerRating           float64
AvgDeliveryTime_Min     float64
Status                 object
dtype: object
```

```
In [40]: df4.isnull().sum()
```

```
Out[40]: PartnerID             0
FirstName             0
LastName              0
PhoneNumber            0
City                  0
AssignedStoreID        0
VehicleType            0
DrivingLicenseNo       0
JoiningDate            0
TotalDeliveries        0
PartnerRating           0
AvgDeliveryTime_Min     0
Status                 0
dtype: int64
```

```
In [41]: df4.duplicated().sum()
```

Out[41]: np.int64(0)

```
In [42]: df5 = pd.read_csv(r"C:\Users\papus\OneDrive\Documents\Swiggy Instamart Project\05_orders.csv")
df5.head()
```

Out[42]:

	OrderID	CustomerID	StoreID	PartnerID	OrderDateTime	DeliveryDateTime	DeliveryTime_Min
0	ORD000001	CUST03967	STORE028	PART0281	02-02-2026 08:50	02-02-2026 08:59	8.80
1	ORD000002	CUST01698	STORE032	PART0706	21-01-2026 05:19	21-01-2026 05:28	8.80
2	ORD000003	CUST04587	STORE016	PART0164	07-02-2026 12:02	07-02-2026 12:35	33.50
3	ORD000004	CUST02083	STORE038	PART0617	07-02-2026 19:41	07-02-2026 20:13	32.10
4	ORD000005	CUST00372	STORE037	PART0379	18-05-2026 05:19	18-05-2026 05:40	21.00



```
In [43]: df5.shape
```

Out[43]: (25000, 14)

```
In [44]: df5.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 25000 entries, 0 to 24999
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   OrderID                25000 non-null  object
1   CustomerID             25000 non-null  object
2   StoreID                25000 non-null  object
3   PartnerID              25000 non-null  object
4   OrderDateTime           25000 non-null  object
5   DeliveryDateTime        25000 non-null  object
6   DeliveryTime_Min        25000 non-null  float64
7   OrderStatus            25000 non-null  object
8   TotalAmount             25000 non-null  float64
9   Distance_KM             25000 non-null  float64
10  PaymentMethod           25000 non-null  object
11  CouponApplied           25000 non-null  object
12  DiscountAmount          25000 non-null  float64
13  FinalAmount             25000 non-null  float64
dtypes: float64(5), object(9)
memory usage: 2.7+ MB
```

```
In [45]: df5["OrderDateTime"] = pd.to_datetime(
df5["OrderDateTime"],
errors="coerce"
)

df5["DeliveryDateTime"] = pd.to_datetime(
df5["DeliveryDateTime"],
errors="coerce"
)
```

```
In [46]: df5.dtypes
```

```
Out[46]: OrderID                object
CustomerID                object
StoreID                   object
PartnerID                 object
OrderDateTime             datetime64[ns]
DeliveryDateTime          datetime64[ns]
DeliveryTime_Min          float64
OrderStatus               object
TotalAmount               float64
Distance_KM               float64
PaymentMethod             object
CouponApplied             object
DiscountAmount            float64
FinalAmount               float64
dtype: object
```

```
In [47]: df5.isnull().sum()
```

```
Out[47]: OrderID                0
CustomerID                0
StoreID                   0
PartnerID                 0
OrderDateTime            15015
DeliveryDateTime          15012
DeliveryTime_Min          0
OrderStatus               0
TotalAmount               0
Distance_KM               0
PaymentMethod             0
CouponApplied             0
DiscountAmount            0
FinalAmount               0
dtype: int64
```

```
In [48]: df5.groupby("OrderStatus")[["OrderDateTime", "DeliveryDateTime"]].apply(
    lambda x: x.isna().sum()
)
```

Out[48]:

	OrderDateTime	DeliveryDateTime
OrderStatus		
Cancelled	1529	1532
Delivered	12743	12737
Returned	743	743

```
In [49]: print(df5["OrderStatus"].value_counts(dropna=False))
```

```
OrderStatus
Delivered    21220
Cancelled    2517
Returned     1263
Name: count, dtype: int64
```

```
In [50]: print(
    df5[df5["OrderDateTime"].isna()][["OrderStatus"]]
    .value_counts()
)
```

```
print(
    df5[df5["DeliveryDateTime"].isna()][ "OrderStatus"]
    .value_counts()
)
```

```
OrderStatus
Delivered    12743
Cancelled    1529
Returned      743
Name: count, dtype: int64
OrderStatus
Delivered    12737
Cancelled    1532
Returned      743
Name: count, dtype: int64
```

```
In [51]: df5[df5["DeliveryDateTime"].isna()][
        ["OrderStatus", "DeliveryTime_Min", "TotalAmount", "FinalAmount"]
        ].describe(include="all")
```

```
Out[51]:
```

	OrderStatus	DeliveryTime_Min	TotalAmount	FinalAmount
count	15012	15012.00	15012.00	15012.00
unique	3	NaN	NaN	NaN
top	Delivered	NaN	NaN	NaN
freq	12737	NaN	NaN	NaN
mean	NaN	21.57	1316.13	1281.84
std	NaN	7.80	683.10	684.99
min	NaN	8.00	150.29	10.45
25%	NaN	14.80	719.90	684.87
50%	NaN	21.50	1313.48	1280.24
75%	NaN	28.40	1906.27	1867.88
max	NaN	35.00	2499.80	2499.41

```
In [52]: df5[df5["OrderDateTime"].isna()][
        ["OrderStatus", "DeliveryTime_Min", "TotalAmount", "FinalAmount"]
        ].describe(include="all")
```

Out[52]:

	OrderStatus	DeliveryTime_Min	TotalAmount	FinalAmount
count	15015	15015.00	15015.00	15015.00
unique	3	NaN	NaN	NaN
top	Delivered	NaN	NaN	NaN
freq	12743	NaN	NaN	NaN
mean	NaN	21.57	1316.10	1281.83
std	NaN	7.80	683.31	685.17
min	NaN	8.00	150.29	10.45
25%	NaN	14.80	719.40	684.26
50%	NaN	21.50	1313.38	1280.13
75%	NaN	28.40	1906.32	1867.91
max	NaN	35.00	2499.80	2499.41

In [53]:

df5.duplicated().sum()

Out[53]: np.int64(0)

In [56]:

df6 = pd.read_csv(r"C:\Users\papus\OneDrive\Documents\Swiggy Instamart Project\06_order_items"
df6.head()

Out[56]:

	OrderItemID	OrderID	ProductID	ProductName	Category	Quantity	PricePerUnit	ItemTotal
0	ITEM000001	ORD000001	PROD00686	Pears Sanitizer 1pc	Personal Care	1	64.87	64.87
1	ITEM000002	ORD000001	PROD00524	Windsor Fresh Rusk 250g	Bakery & Bread	2	8.93	17.86
2	ITEM000003	ORD000001	PROD00253	Whiskas Organic Pet Accessories 1pc	Pet Care	1	38.32	38.32
3	ITEM000004	ORD000001	PROD00906	Drools Organic Pet Food 2kg	Pet Care	3	16.45	49.34
4	ITEM000005	ORD000001	PROD00621	Nestle Fresh Baby Food 500g	Baby Care	1	42.60	42.60

In [58]:

df6.shape

Out[58]: (87559, 8)

In [59]:

df6.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 87559 entries, 0 to 87558
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   OrderItemID     87559 non-null  object
1   OrderID         87559 non-null  object
2   ProductID       87559 non-null  object
3   ProductName     87559 non-null  object
4   Category        87559 non-null  object
5   Quantity        87559 non-null  int64
6   PricePerUnit    87559 non-null  float64
7   ItemTotal       87559 non-null  float64
dtypes: float64(2), int64(1), object(5)
memory usage: 5.3+ MB
```

```
In [60]: df6.isnull().sum()
```

```
Out[60]: OrderItemID    0
OrderID        0
ProductID      0
ProductName     0
Category       0
Quantity       0
PricePerUnit   0
ItemTotal      0
dtype: int64
```

```
In [61]: df6.duplicated().sum()
```

```
Out[61]: np.int64(0)
```

```
In [62]: print(df6.columns.tolist())
```

```
['OrderItemID', 'OrderID', 'ProductID', 'ProductName', 'Category', 'Quantity', 'PricePerUnit',
'ItemTotal']
```

```
In [63]: df7 = pd.read_csv(r"C:\Users\papus\OneDrive\Documents\Swiggy Instamart Project\07_payments.csv")
df7.head()
```

```
Out[63]:
```

	PaymentID	OrderID	TransactionID	PaymentMethod	PaymentGateway	TransactionAmount
0	PAY000001	ORD000001	TXN330056618729	Credit Card	PhonePe	225.10
1	PAY000002	ORD000002	TXN629018471067	Instamart Pay	Razorpay	1592.82
2	PAY000003	ORD000003	TXN577671124770	Credit Card	Razorpay	2447.26
3	PAY000004	ORD000004	TXN250702835871	UPI	Paytm	573.02
4	PAY000005	ORD000005	TXN675331637352	Debit Card	Razorpay	1325.93

```
In [65]: df7.shape
```

```
Out[65]: (25000, 10)
```

```
In [67]: print(df7.columns.tolist())
```

```
['PaymentID', 'OrderID', 'TransactionID', 'PaymentMethod', 'PaymentGateway', 'TransactionAmount',
'PaymentStatus', 'PaymentDateTime', 'RefundAmount', 'RefundDateTime']
```

```
In [66]: df7.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 25000 entries, 0 to 24999
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   PaymentID              25000 non-null  object
1   OrderID                 25000 non-null  object
2   TransactionID           25000 non-null  object
3   PaymentMethod           25000 non-null  object
4   PaymentGateway          25000 non-null  object
5   TransactionAmount       25000 non-null  float64
6   PaymentStatus           25000 non-null  object
7   PaymentDateTime         25000 non-null  object
8   RefundAmount            25000 non-null  float64
9   RefundDateTime          2950 non-null   object
dtypes: float64(2), object(8)
memory usage: 1.9+ MB
```

```
In [68]: df7["PaymentDateTime"] = pd.to_datetime(
          df7["PaymentDateTime"],
          errors="coerce"
        )

df7["RefundDateTime"] = pd.to_datetime(
    df7["RefundDateTime"],
    errors="coerce"
)
```

C:\Users\papus\AppData\Local\Temp\ipykernel_12980\2917812957.py:6: UserWarning: Parsing dates in %d-%m-%Y %H:%M format when dayfirst=False (the default) was specified. Pass `dayfirst=True` or specify a format to silence this warning.

```
df7["RefundDateTime"] = pd.to_datetime(
```

```
In [69]: df7.dtypes
```

```
Out[69]: PaymentID              object
OrderID                 object
TransactionID           object
PaymentMethod           object
PaymentGateway          object
TransactionAmount       float64
PaymentStatus           object
PaymentDateTime         datetime64[ns]
RefundAmount            float64
RefundDateTime          datetime64[ns]
dtype: object
```

```
In [70]: df7.isnull().sum()
```

```
Out[70]: PaymentID              0
OrderID                 0
TransactionID           0
PaymentMethod           0
PaymentGateway          0
TransactionAmount       0
PaymentStatus           0
PaymentDateTime         15012
RefundAmount            0
RefundDateTime          22050
dtype: int64
```

```
In [71]: df7[df7["PaymentDateTime"].isna()][["PaymentStatus"].value_counts(dropna=False)
```

```
Out[71]: PaymentStatus
Success      12230
Refunded      1797
Failed         985
Name: count, dtype: int64

In [72]: df7[df7["RefundDateTime"].isna()]["PaymentStatus"].value_counts(dropna=False)

Out[72]: PaymentStatus
Success      20369
Failed       1681
Name: count, dtype: int64

In [73]: df7.duplicated().sum()

Out[73]: np.int64(0)

In [74]: df8 = pd.read_csv(r"C:\Users\papus\OneDrive\Documents\Swiggy Instamart Project\08_inventory.csv")
df8.shape

Out[74]: (25000, 9)

In [75]: df8.head()
```

	InventoryID	StoreID	ProductID	StockLevel	ReorderLevel	StockStatus	LastRestockedDate	CapacityPercent
0	INV000001	STORE014	PROD00015	412	48	In Stock	13-07-2026	100
1	INV000002	STORE010	PROD00952	47	41	In Stock	01-07-2026	100
2	INV000003	STORE016	PROD00186	79	29	In Stock	23-06-2026	100
3	INV000004	STORE038	PROD00678	89	24	In Stock	30-06-2026	100
4	INV000005	STORE012	PROD00620	73	23	In Stock	05-07-2026	100

```
In [76]: df8.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 25000 entries, 0 to 24999
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   InventoryID            25000 non-null  object
1   StoreID                25000 non-null  object
2   ProductID              25000 non-null  object
3   StockLevel             25000 non-null  int64
4   ReorderLevel           25000 non-null  int64
5   StockStatus            25000 non-null  object
6   LastRestockedDate      25000 non-null  object
7   CapacityPercent        25000 non-null  float64
8   LastUpdated            25000 non-null  object
dtypes: float64(1), int64(2), object(6)
memory usage: 1.7+ MB

In [77]: df8["LastRestockedDate"] = pd.to_datetime(
df8["LastRestockedDate"],
```

```

        format="%d-%m-%Y",
        errors="coerce"
    )

    df8["LastUpdated"] = pd.to_datetime(
        df8["LastUpdated"],
        format="%d-%m-%Y",
        errors="coerce"
    )

```

In [78]: `df8.dtypes`

```

Out[78]: InventoryID          object
StoreID          object
ProductID        object
StockLevel       int64
ReorderLevel     int64
StockStatus      object
LastRestockedDate  datetime64[ns]
CapacityPercent   float64
LastUpdated      datetime64[ns]
dtype: object

```

In [79]: `df8.isnull().sum()`

```

Out[79]: InventoryID          0
StoreID          0
ProductID        0
StockLevel       0
ReorderLevel     0
StockStatus      0
LastRestockedDate 0
CapacityPercent   0
LastUpdated      25000
dtype: int64

```

In [80]: `df8.duplicated().sum()`

Out[80]: `np.int64(0)`

In [82]: `df9 = pd.read_csv(r"C:\Users\papus\OneDrive\Documents\Swiggy Instamart Project\09_offers.csv")`
`df9.shape`

Out[82]: `(14953, 10)`

In [83]: `df9.head()`

Out [83]:

	OfferID	OrderID	OfferName	OfferType	DiscountAmount	SalesGenerated	RedemptionRate
0	OFFER00001	ORD000001	â,125 Cashback	Cashback	100.76	225.10	39.10
1	OFFER00002	ORD000002	Flat 20% Off	Discount	237.46	1592.82	49.80
2	OFFER00003	ORD000003	â,125 Cashback	Cashback	70.46	2447.26	36.30
3	OFFER00004	ORD000004	10% Off using SBI Cards	Bank Offer	116.46	573.02	51.40
4	OFFER00005	ORD000007	â,1100 Off on ICICI Cards	Bank Offer	99.23	1797.68	72.30

In [84]:

```
print(df9.columns.tolist())
```

['OfferID', 'OrderID', 'OfferName', 'OfferType', 'DiscountAmount', 'SalesGenerated', 'RedemptionRate', 'ROI', 'OfferDate', 'Status']

In [85]:

```
df9["OfferDate"] = pd.to_datetime(
    df9["OfferDate"],
    format="%d-%m-%Y",
    errors="coerce"
)
```

In [86]:

```
df9.info()
```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14953 entries, 0 to 14952
Data columns (total 10 columns):
Column Non-Null Count Dtype
--- ---
0 OfferID 14953 non-null object
1 OrderID 14953 non-null object
2 OfferName 14953 non-null object
3 OfferType 14953 non-null object
4 DiscountAmount 14953 non-null float64
5 SalesGenerated 14953 non-null float64
6 RedemptionRate 14953 non-null float64
7 ROI 14953 non-null float64
8 OfferDate 0 non-null datetime64[ns]
9 Status 14953 non-null object
dtypes: datetime64[ns](1), float64(4), object(5)
memory usage: 1.1+ MB

In [87]:

```
df9.isnull().sum()
```

```
Out[87]: OfferID          0
        OrderID          0
        OfferName        0
        OfferType        0
        DiscountAmount    0
        SalesGenerated    0
        RedemptionRate    0
        ROI              0
        OfferDate        14953
        Status           0
        dtype: int64
```

```
In [88]: df9.duplicated().sum()
```

```
Out[88]: np.int64(0)
```

Revenue Trends

Revenue Fields

```
In [89]: # Checking revenue-related columns
print(df5[["OrderID",
            "OrderDateTime",
            "OrderStatus",
            "TotalAmount",
            "DiscountAmount",
            "FinalAmount"]].head())
```

	OrderID	OrderDateTime	OrderStatus	TotalAmount	DiscountAmount	\
0	ORD000001	2026-02-02 08:50:00	Delivered	225.10	0.00	
1	ORD000002	NaT	Delivered	1650.31	57.49	
2	ORD000003	2026-07-02 12:02:00	Delivered	2447.26	0.00	
3	ORD000004	2026-07-02 19:41:00	Delivered	573.02	0.00	
4	ORD000005	NaT	Returned	1325.93	0.00	

	FinalAmount
0	225.10
1	1592.82
2	2447.26
3	573.02
4	1325.93

```
In [90]: # Summary statistics
df5[["TotalAmount",
      "DiscountAmount",
      "FinalAmount"]].describe()
```

Out[90]:

	TotalAmount	DiscountAmount	FinalAmount
count	25000.00	25000.00	25000.00
mean	1319.32	34.50	1284.82
std	680.01	48.46	681.77
min	150.19	0.00	10.45
25%	730.62	0.00	693.37
50%	1318.14	0.00	1282.28
75%	1905.81	69.85	1870.31
max	2499.80	150.00	2499.73

Basic revenue metrics

```
In [91]: total_revenue = df5["FinalAmount"].sum()
total_gross_amount = df5["TotalAmount"].sum()
total_discount = df5["DiscountAmount"].sum()

print(f"Total Gross Amount: ₹{total_gross_amount:,.2f}")
print(f"Total Discount: ₹{total_discount:,.2f}")
print(f"Total Final Revenue: ₹{total_revenue:,.2f}")
```

Total Gross Amount: ₹32,982,940.96
Total Discount: ₹862,417.55
Total Final Revenue: ₹32,120,523.41

Revenue by order status

```
In [92]: revenue_by_status = (
    df5.groupby("OrderStatus")["FinalAmount"]
    .agg(["count", "sum", "mean"])
    .round(2)
)

print(revenue_by_status)
```

	count	sum	mean
OrderStatus			
Cancelled	2517	3247018.68	1290.04
Delivered	21220	27259102.27	1284.59
Returned	1263	1614402.46	1278.23

Creating an analysis dataset

```
In [93]: df5_revenue = df5[
    df5["OrderStatus"] == "Delivered"
].copy()

print("Delivered Orders:", len(df5_revenue))
```

Delivered Orders: 21220

```
In [94]: delivered_revenue = df5_revenue["FinalAmount"].sum()

print(f"Delivered Revenue: ₹{delivered_revenue:,.2f}")
```

Delivered Revenue: ₹27,259,102.27

Daily Revenue Trend

```
In [95]: df5_revenue = df5_revenue.dropna(
          subset=["OrderDateTime"]
        ).copy()

df5_revenue["OrderDate"] = (
    df5_revenue["OrderDateTime"].dt.date
)

daily_revenue = (
    df5_revenue
    .groupby("OrderDate")["FinalAmount"]
    .sum()
    .reset_index()
)

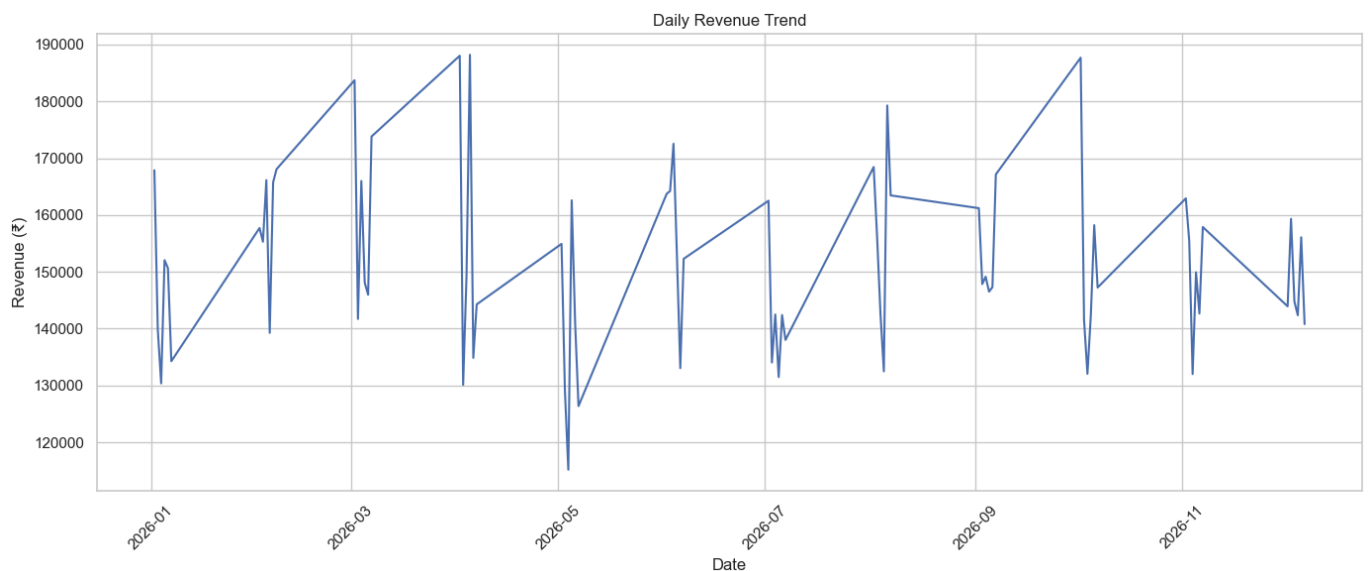
print(daily_revenue.head())
```

	OrderDate	FinalAmount
0	2026-01-02	167878.65
1	2026-01-03	139907.25
2	2026-01-04	130373.46
3	2026-01-05	152057.62
4	2026-01-06	150661.48

```
In [96]: plt.figure(figsize=(14, 6))

sns.lineplot(
    data=daily_revenue,
    x="OrderDate",
    y="FinalAmount"
)

plt.title("Daily Revenue Trend")
plt.xlabel("Date")
plt.ylabel("Revenue (₹)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



Revenue by City

```
In [97]: revenue_by_city = (
    df5_revenue
    .merge(
        df2[["StoreID", "City"]],
        on="StoreID",
        how="left"
    )
    .groupby("City")["FinalAmount"]
    .sum()
    .sort_values(ascending=False)
)

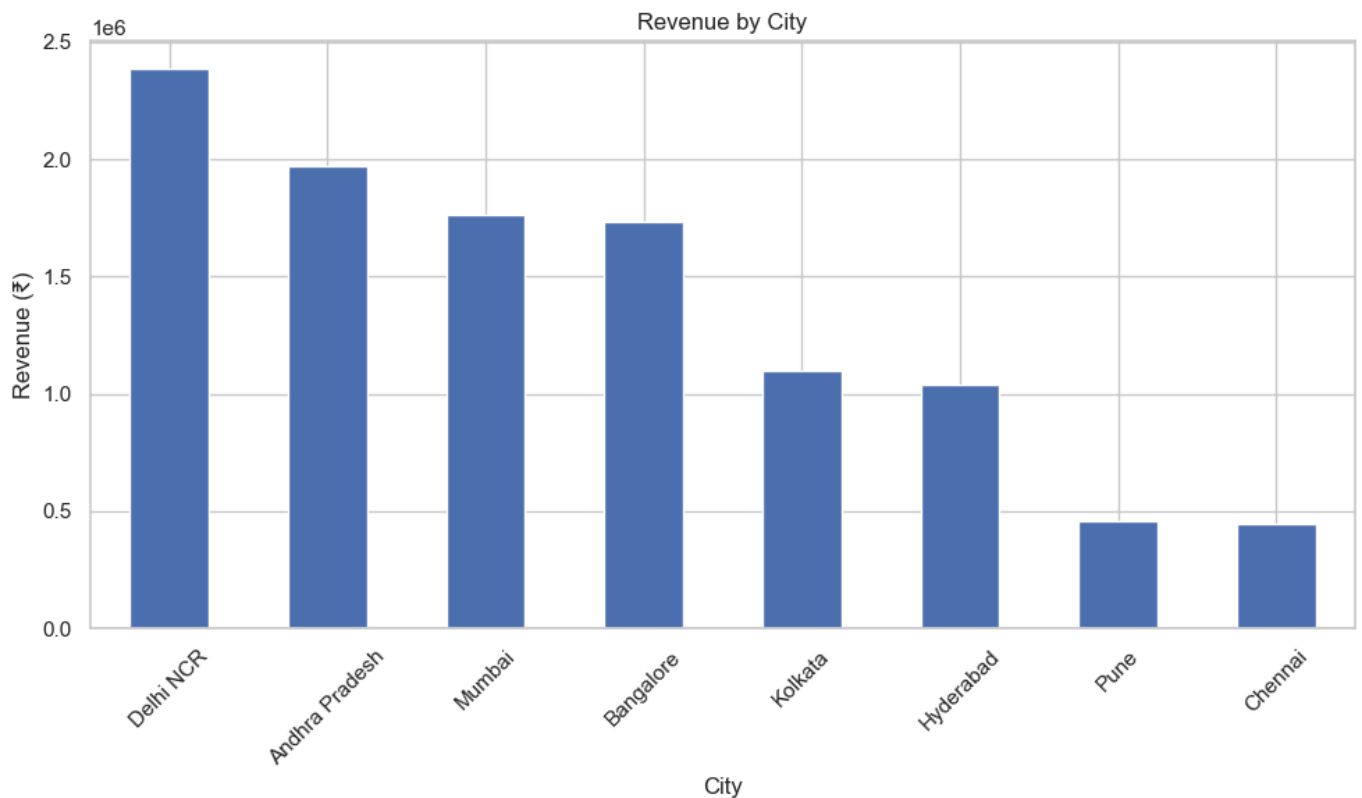
print(revenue_by_city)
```

```
City
Delhi NCR      2383902.78
Andhra Pradesh 1971640.22
Mumbai         1763241.24
Bangalore      1732723.29
Kolkata         1101077.49
Hyderabad       1040323.03
Pune           459208.84
Chennai         447480.36
Name: FinalAmount, dtype: float64
```

```
In [98]: plt.figure(figsize=(10, 6))

revenue_by_city.plot(kind="bar")

plt.title("Revenue by City")
plt.xlabel("City")
plt.ylabel("Revenue (₹)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



Revenue by Category

```
In [99]: category_revenue = (
    df6
    .merge(
        df5_revenue[["OrderID"]],
        on="OrderID",
        how="inner"
    )
    .groupby("Category")["ItemTotal"]
    .sum()
    .sort_values(ascending=False)
)

print(category_revenue)
```

```
Category
Atta & Grains      1185003.91
Beverages          1162534.94
Oil & Masalas      1093102.76
Household          1045731.68
Snacks & Munchies  1016084.55
Personal Care      992165.99
Pet Care           979153.49
Dairy & Eggs       969825.86
Fruits & Vegetables 940849.78
Bakery & Bread     933488.35
Baby Care          874730.89
Name: ItemTotal, dtype: float64
```

Revenue Trends Analysis

Objective

Analyze revenue performance across the Q-Commerce business to understand total revenue, discounts, order-status contribution, daily revenue trends, geographic revenue distribution, and product-category contribution.

Methodology

The analysis uses:

- **Orders (df5)** for order-level revenue and transaction analysis.
- **Stores (df2)** to map orders to cities.
- **Order Items (df6)** to analyze revenue contribution by product category.

For realized revenue analysis, only orders with `OrderStatus = "Delivered"` were treated as revenue-generating orders.

Overall Revenue Metrics

Metric	Amount
Total Gross Amount	₹32,982,940.96
Total Discount	₹862,417.55

Metric	Amount
Total Final Amount	₹32,120,523.41
Delivered Revenue	₹27,259,102.27

The gross amount reconciles with the final amount:

Gross Amount – Discount = Final Amount

₹32,982,940.96 – ₹862,417.55 = ₹32,120,523.41

This confirms that the revenue calculation is internally consistent.

Revenue by Order Status

Order Status	Orders	Revenue	Average Order Value
Delivered	21,220	₹27,259,102.27	₹1,284.59
Cancelled	2,517	₹3,247,018.68	₹1,290.04
Returned	1,263	₹1,614,402.46	₹1,278.23

For operational revenue analysis, **Delivered orders** were used as the primary revenue population because cancelled and returned orders do not represent realized sales in the same way as completed deliveries.

Daily Revenue Trend

Daily revenue was calculated using delivered orders with valid `OrderDateTime` values.

Example daily revenue observations:

Date	Revenue
2026-01-02	₹167,878.65
2026-01-03	₹139,907.25
2026-01-04	₹130,373.46
2026-01-05	₹152,057.62
2026-01-06	₹150,661.48

A line chart was used to visualize daily revenue movement over time and identify changes in revenue activity across the analysis period.

Revenue by City

City	Delivered Revenue
Delhi NCR	₹2,383,902.78
Andhra Pradesh	₹1,971,640.22

City	Delivered Revenue
Mumbai	₹1,763,241.24
Bangalore	₹1,732,723.29
Kolkata	₹1,101,077.49
Hyderabad	₹1,040,323.03
Pune	₹459,208.84
Chennai	₹447,480.36

Revenue varies substantially across markets. Delhi NCR recorded the highest delivered revenue in the analyzed city-level results, while Chennai recorded the lowest.

These differences should be interpreted alongside market size, order volume, store coverage, and fleet capacity rather than revenue alone.

Revenue by Product Category

Product Category	Revenue
Atta & Grains	₹1,185,003.91
Beverages	₹1,162,534.94
Oil & Masalas	₹1,093,102.76
Household	₹1,045,731.68
Snacks & Munchies	₹1,016,084.55
Personal Care	₹992,165.99
Pet Care	₹979,153.49
Dairy & Eggs	₹969,825.86
Fruits & Vegetables	₹940,849.78
Bakery & Bread	₹933,488.35
Baby Care	₹874,730.89

Atta & Grains generated the highest category-level revenue at **₹1.19 million**, followed by Beverages and Oil & Masalas.

Business Insights

- The dataset generated **₹32.12 million in total final transaction value**.
- Delivered orders contributed **₹27.26 million** in revenue.
- **21,220 orders** were delivered during the analysis period.
- Discounts reduced gross transaction value by approximately **₹0.86 million**.
- Revenue varies significantly across cities, highlighting differences in market-level demand and scale.
- Atta & Grains was the highest-revenue product category, while Baby Care generated the lowest revenue among the analyzed categories.

- Daily revenue analysis provides a foundation for the later **Demand Forecasting** component of the project.

Conclusion

The Revenue Trends analysis establishes the financial baseline for the Q-Commerce operation. It identifies realized revenue, discount impact, geographic revenue distribution, and category-level contribution. These findings will support subsequent analyses such as **CLV Prediction, Demand Forecasting, Inventory Replenishment, and Dark Store Capacity Planning**.

Fleet Utilization

Fleet Overview

In [101...

```
# Fleet overview
print("Number of Delivery Partners:", df4["PartnerID"].nunique())

print("\nVehicle Type Distribution:")
print(df4["VehicleType"].value_counts())

print("\nPartner Status Distribution:")
print(df4["Status"].value_counts())

print("\nFleet by City:")
print(df4["City"].value_counts())
```

Number of Delivery Partners: 800

Vehicle Type Distribution:

VehicleType

Bike	382
Scooter	249
Electric Vehicle (EV)	127
Bicycle	42

Name: count, dtype: int64

Partner Status Distribution:

Status

Active	644
On Leave	78
Inactive	65
Suspended	13

Name: count, dtype: int64

Fleet by City:

City

Mumbai	164
Bangalore	146
Andhra Pradesh	135
Hyderabad	118
Delhi NCR	110
Kolkata	48
Chennai	41
Pune	38

Name: count, dtype: int64

Actual Fleet Utilization

In [102...

```
# Delivery partner utilization
partner_utilization = (
    df5[df5["OrderStatus"] == "Delivered"]
    .groupby("PartnerID")
    .agg(
        Total_Deliveries=("OrderID", "count"),
        Total_Revenue=("FinalAmount", "sum"),
        Avg_Delivery_Time=("DeliveryTime_Min", "mean")
    )
    .reset_index()
)

print(partner_utilization.head())
print("\nNumber of partners with delivered orders:",
      partner_utilization["PartnerID"].nunique())
```

	PartnerID	Total_Deliveries	Total_Revenue	Avg_Delivery_Time
0	PART0001	19	24562.09	22.18
1	PART0002	34	49854.67	21.05
2	PART0003	29	41776.31	20.35
3	PART0004	27	35187.05	17.43
4	PART0005	42	54769.28	20.84

Number of partners with delivered orders: 800

Fleet Utilization Summary

In [103...

```
fleet_utilization_summary = partner_utilization[
    ["Total_Deliveries", "Total_Revenue", "Avg_Delivery_Time"]
].describe()

print(fleet_utilization_summary)
```

	Total_Deliveries	Total_Revenue	Avg_Delivery_Time
count	800.00	800.00	800.00
mean	26.52	34073.88	21.58
std	5.06	7220.12	1.52
min	11.00	13421.25	15.59
25%	23.00	29067.68	20.56
50%	26.00	33767.81	21.51
75%	30.00	38432.99	22.65
max	42.00	55212.49	26.12

Utilization by Vehicle Type

In [104...

```
fleet_by_vehicle = (
    partner_utilization
    .merge(
        df4[["PartnerID", "VehicleType"]],
        on="PartnerID",
        how="left"
    )
    .groupby("VehicleType")
    .agg(
        Partners=("PartnerID", "nunique"),
        Total_Deliveries=("Total_Deliveries", "sum"),
        Avg_Deliveries=("Total_Deliveries", "mean"),
        Avg_Delivery_Time=("Avg_Delivery_Time", "mean"),
        Total_Revenue=("Total_Revenue", "sum")
    )
    .round(2)
    .sort_values("Avg_Deliveries", ascending=False)
```

```
)

print(fleet_by_vehicle)
```

	Partners	Total_Deliveries	Avg_Deliveries	\
VehicleType				
Electric Vehicle (EV)	127	3415	26.89	
Bicycle	42	1118	26.62	
Bike	382	10158	26.59	
Scooter	249	6529	26.22	

	Avg_Delivery_Time	Total_Revenue
VehicleType		
Electric Vehicle (EV)	21.70	4349890.47
Bicycle	21.59	1435312.75
Bike	21.56	13014875.00
Scooter	21.54	8459024.05

Fleet Utilization by City

In [105...

```
fleet_by_city = (
    partner_utilization
    .merge(
        df4[["PartnerID", "City"]],
        on="PartnerID",
        how="left"
    )
    .groupby("City")
    .agg(
        Partners=("PartnerID", "nunique"),
        Total_Deliveries=("Total_Deliveries", "sum"),
        Avg_Deliveries=("Total_Deliveries", "mean"),
        Avg_Delivery_Time=("Avg_Delivery_Time", "mean"),
        Total_Revenue=("Total_Revenue", "sum")
    )
    .round(2)
    .sort_values("Avg_Deliveries", ascending=False)
)

print(fleet_by_city)
```

	Partners	Total_Deliveries	Avg_Deliveries	Avg_Delivery_Time	\
City					
Pune	38	1067	28.08	21.28	
Chennai	41	1116	27.22	21.47	
Kolkata	48	1303	27.15	21.84	
Bangalore	146	3921	26.86	21.64	
Mumbai	164	4379	26.70	21.68	
Andhra Pradesh	135	3563	26.39	21.56	
Delhi NCR	110	2838	25.80	21.40	
Hyderabad	118	3033	25.70	21.59	

	Total_Revenue
City	
Pune	1388446.03
Chennai	1420473.44
Kolkata	1674688.40
Bangalore	5010975.11
Mumbai	5605445.82
Andhra Pradesh	4583691.58
Delhi NCR	3667449.77
Hyderabad	3907932.12

Fleet Utilization Rate

In [106...]

```
active_partners = df4[df4["Status"] == "Active"]["PartnerID"].nunique()

delivered_orders = df5[
    df5["OrderStatus"] == "Delivered"
]["OrderID"].nunique()

fleet_utilization_rate = delivered_orders / active_partners

print("Active Delivery Partners:", active_partners)
print("Delivered Orders:", delivered_orders)
print("Delivered Orders per Active Partner:", round(fleet_utilization_rate, 2))
```

Active Delivery Partners: 644

Delivered Orders: 21220

Delivered Orders per Active Partner: 32.95

Fleet Utilization Analysis

Objective

Analyze delivery-partner utilization across the Q-Commerce network to understand fleet workload, vehicle-type performance, geographic distribution, and workload per active delivery partner.

Methodology

The analysis combines:

- **Delivery Partners (df4)** — partner details, vehicle type, status, and city.
- **Orders (df5)** — delivered orders, revenue, and delivery time.

Only orders with `OrderStatus = "Delivered"` were used for actual delivery workload and revenue analysis.

Key Metrics

The analysis evaluated:

1. Total delivery-partner fleet size
2. Active delivery-partner count
3. Delivered orders handled by each partner
4. Revenue handled by each partner
5. Average delivery time
6. Fleet workload by vehicle type
7. Fleet workload by city
8. Delivered orders per active delivery partner

Fleet Overview

The dataset contains **800 delivery partners**, of which **644 are currently active**.

Vehicle Fleet

Vehicle Type	Partners
Bike	382
Scooter	249
Electric Vehicle (EV)	127
Bicycle	42

Partner Status

Status	Partners
Active	644
On Leave	78
Inactive	65
Suspended	13

Partner-Level Utilization

All 800 delivery partners handled at least one delivered order during the analysis period.

Metric	Result
Average deliveries per partner	26.52
Median deliveries per partner	26
Minimum deliveries	11
Maximum deliveries	42
Average revenue handled per partner	₹34,073.88
Average delivery time	21.58 minutes

The relatively narrow range around the median indicates that delivered-order workload is distributed fairly consistently across the delivery-partner fleet.

Utilization by Vehicle Type

Vehicle Type	Partners	Total Deliveries	Avg. Deliveries/Partner	Avg. Delivery Time
Electric Vehicle (EV)	127	3,415	26.89	21.70 min
Bicycle	42	1,118	26.62	21.59 min
Bike	382	10,158	26.59	21.56 min
Scooter	249	6,529	26.22	21.54 min

Average workload was very similar across vehicle types, ranging from **26.22 to 26.89 deliveries per partner**. Therefore, vehicle type alone does not show a major difference in individual delivery workload in this dataset.

Bike partners handled the largest total number of deliveries because bikes represent the largest fleet segment.

Utilization by City

City	Partners	Avg. Deliveries/Partner	Avg. Delivery Time
Pune	38	28.08	21.28 min
Chennai	41	27.22	21.47 min
Kolkata	48	27.15	21.84 min
Bangalore	146	26.86	21.64 min
Mumbai	164	26.70	21.68 min
Andhra Pradesh	135	26.39	21.56 min
Delhi NCR	110	25.80	21.40 min
Hyderabad	118	25.70	21.59 min

Pune recorded the highest average workload at **28.08 deliveries per partner**, while Hyderabad recorded the lowest at **25.70 deliveries per partner**.

The difference indicates some variation in fleet workload across markets, although average delivery times remained relatively stable at approximately 21–22 minutes.

Active Fleet Workload

There were **21,220 delivered orders** handled by **644 active delivery partners**.

This resulted in:

32.95 delivered orders per active delivery partner over the full analysis period.

This metric represents the workload ratio for the complete dataset period rather than a percentage utilization rate.

Business Insights

- The delivery network has **800 partners**, with **644 currently active**.
- Every delivery partner handled at least one delivered order during the analysis period.
- Average workload across all partners was **26.52 deliveries per partner**.
- Vehicle-level workload was highly consistent, with only a small difference between vehicle types.
- Fleet workload varied more across cities, with Pune having the highest average deliveries per partner and Hyderabad the lowest.
- Average delivery time remained relatively stable across both vehicle types and cities.
- The analysis provides a baseline for future **fleet mix optimization**, where vehicle composition can be evaluated against demand, delivery time, and operational requirements.

CLV Prediction

Understanding the Customer Data

In [107... *# Customer dataset overview*

```
print("Shape:", df3.shape)

print("\nColumns:")
print(df3.columns.tolist())

print("\nCustomer sample:")
print(df3.head())
```

Shape: (5000, 14)

Columns:

['CustomerID', 'FirstName', 'LastName', 'Email', 'PhoneNumber', 'City', 'Pincode', 'DateOfBirth', 'Gender', 'RegistrationDate', 'LoyaltyTier', 'TotalLifetimeValue', 'TotalOrdersPlaced', 'IsActive']

Customer sample:

	CustomerID	FirstName	LastName	Email	PhoneNumber	\
0	CUST00001	Isaac	Bakshi	isaac.bakshi60@email.com	9944499530	
1	CUST00002	Azad	Balan	azad.balan42@email.com	9772666902	
2	CUST00003	Oni	Kannan	oni.kannan29@email.com	9443072390	
3	CUST00004	Tanveer	Nayar	tanveer.nayar36@email.com	9451517302	
4	CUST00005	Caleb	Sharaf	caleb.sharaf55@email.com	9614117732	

	City	Pincode	DateOfBirth	Gender	RegistrationDate	LoyaltyTier	\
0	Mumbai	332181	2000-09-23	Other	2025-06-10	Bronze	
1	Delhi NCR	338908	1974-01-29	Other	2026-01-25	Bronze	
2	Bangalore	940265	1979-06-16	Female	2024-11-15	Gold	
3	Bangalore	161559	1978-07-19	Other	2026-04-05	Gold	
4	Andhra Pradesh	618495	1999-11-02	Female	2024-12-14	Bronze	

	TotalLifetimeValue	TotalOrdersPlaced	IsActive
0	4627.01	8	1
1	4283.42	10	1
2	23389.64	150	0
3	16445.55	88	1
4	3072.52	17	1

Validating Customer Metrics

In [108... *# Validate existing customer-level CLV metrics*

```
print("Total Lifetime Value:")
print(df3["TotalLifetimeValue"].describe())

print("\nTotal Orders Placed:")
print(df3["TotalOrdersPlaced"].describe())

print("\nLoyalty Tier:")
print(df3["LoyaltyTier"].value_counts())

print("\nActive Status:")
print(df3["IsActive"].value_counts())
```

```
Total Lifetime Value:
count    5000.00
mean     10612.06
std      12848.19
min       502.39
25%      2778.91
50%      5036.44
75%     13150.67
max      74754.91
Name: TotalLifetimeValue, dtype: float64
```

```
Total Orders Placed:
count    5000.00
mean       43.39
std        44.84
min         1.00
25%        10.00
50%        18.00
75%        77.00
max       150.00
Name: TotalOrdersPlaced, dtype: float64
```

```
Loyalty Tier:
LoyaltyTier
Bronze      2496
Silver      1536
Gold         750
Platinum     218
Name: count, dtype: int64
```

```
Active Status:
IsActive
1      4308
0       692
Name: count, dtype: int64
```

Building Customer Purchase Behavior

In [109...

```
# Building customer-level purchase behavior from delivered orders

customer_orders = (
    df5[df5["OrderStatus"] == "Delivered"]
    .groupby("CustomerID")
    .agg(
        Delivered_Orders=("OrderID", "nunique"),
        Total_Revenue=("FinalAmount", "sum"),
        Avg_Order_Value=("FinalAmount", "mean")
    )
    .reset_index()
)

print(customer_orders.head())

print("\nCustomer records:", len(customer_orders))
```

	CustomerID	Delivered_Orders	Total_Revenue	Avg_Order_Value
0	CUST00001	5	5824.55	1164.91
1	CUST00002	9	12822.93	1424.77
2	CUST00003	4	4615.06	1153.77
3	CUST00004	8	9809.07	1226.13
4	CUST00005	5	7248.96	1449.79

Customer records: 4928

Recency & Tenure

In [110... *# Reference date from the latest valid delivered order*

```
reference_date = df5.loc[
    (df5["OrderStatus"] == "Delivered") &
    (df5["OrderDateTime"].notna()),
    "OrderDateTime"
].max()

print("Reference Date:", reference_date)
```

Reference Date: 2026-12-07 23:52:00

Customer Recency

In [111... *# Calculating customer recency*

```
last_purchase = (
    df5[
        (df5["OrderStatus"] == "Delivered") &
        (df5["OrderDateTime"].notna())
    ]
    .groupby("CustomerID")["OrderDateTime"]
    .max()
    .reset_index(name="Last_Purchase_Date")
)

last_purchase["Recency_Days"] = (
    reference_date - last_purchase["Last_Purchase_Date"]
).dt.days

print(last_purchase.head())

print("\nRecency Summary:")
print(last_purchase["Recency_Days"].describe())
```

	CustomerID	Last_Purchase_Date	Recency_Days
0	CUST00001	2026-09-04 08:21:00	94
1	CUST00002	2026-12-07 04:32:00	0
2	CUST00004	2026-11-07 20:29:00	30
3	CUST00005	2026-07-03 08:54:00	157
4	CUST00006	2026-03-04 00:16:00	278

Recency Summary:

```
count    4089.00
mean      120.74
std        97.56
min         0.00
25%        33.00
50%        95.00
75%       187.00
max       339.00
```

Name: Recency_Days, dtype: float64

Customer Tenure

In [112...

```
# Calculate customer tenure

customer_tenure = df3[[
    "CustomerID",
    "RegistrationDate"
]].copy()

customer_tenure["Tenure_Days"] = (
    reference_date - customer_tenure["RegistrationDate"]
).dt.days

print(customer_tenure.head())

print("\nTenure Summary:")
print(customer_tenure["Tenure_Days"].describe())
```

	CustomerID	RegistrationDate	Tenure_Days
0	CUST00001	2025-06-10	545
1	CUST00002	2026-01-25	316
2	CUST00003	2024-11-15	752
3	CUST00004	2026-04-05	246
4	CUST00005	2024-12-14	723

Tenure Summary:

```
count    5000.00
mean      507.62
std       209.21
min       142.00
25%       330.75
50%       510.00
75%       688.00
max       872.00
```

Name: Tenure_Days, dtype: float64

CLV Feature Dataset

In [113...

```
# Building customer-level CLV feature dataset

clv_data = (
    df3[
        [
            "CustomerID",
```

```

        "LoyaltyTier",
        "IsActive"
    ]
]
.merge(customer_orders, on="CustomerID", how="left")
.merge(last_purchase, on="CustomerID", how="left")
.merge(
    customer_tenure[["CustomerID", "Tenure_Days"]],
    on="CustomerID",
    how="left"
)
)

print("Shape:", clv_data.shape)

print("\nColumns:")
print(clv_data.columns.tolist())

print("\nSample:")
print(clv_data.head())

```

Shape: (5000, 9)

Columns:

['CustomerID', 'LoyaltyTier', 'IsActive', 'Delivered_Orders', 'Total_Revenue', 'Avg_Order_Value', 'Last_Purchase_Date', 'Recency_Days', 'Tenure_Days']

Sample:

	CustomerID	LoyaltyTier	IsActive	Delivered_Orders	Total_Revenue	\
0	CUST00001	Bronze	1	5.00	5824.55	
1	CUST00002	Bronze	1	9.00	12822.93	
2	CUST00003	Gold	0	4.00	4615.06	
3	CUST00004	Gold	1	8.00	9809.07	
4	CUST00005	Bronze	1	5.00	7248.96	

	Avg_Order_Value	Last_Purchase_Date	Recency_Days	Tenure_Days
0	1164.91	2026-09-04 08:21:00	94.00	545
1	1424.77	2026-12-07 04:32:00	0.00	316
2	1153.77	NaT	NaN	752
3	1226.13	2026-11-07 20:29:00	30.00	246
4	1449.79	2026-07-03 08:54:00	157.00	723

In [114... print(clv_data.isnull().sum())

```

CustomerID      0
LoyaltyTier     0
IsActive        0
Delivered_Orders 72
Total_Revenue   72
Avg_Order_Value 72
Last_Purchase_Date 911
Recency_Days    911
Tenure_Days     0
dtype: int64

```

In [115... *# Creating a flag for customers without a valid purchase date*

```

clv_data["No_Purchase_Date"] = clv_data["Recency_Days"].isna().astype(int)

print(
    clv_data["No_Purchase_Date"].value_counts()
)

```

```
No_Purchase_Date
0    4089
1     911
Name: count, dtype: int64
```

In [116...

```
# Comparing customers with and without delivered-order activity

print(
    clv_data.groupby("No_Purchase_Date")[
        ["Delivered_Orders", "Total_Revenue"]
    ].count()
)

print("\nDelivered-order totals:")
print(
    clv_data.groupby("No_Purchase_Date")[
        ["Delivered_Orders", "Total_Revenue"]
    ].sum()
)
```

No_Purchase_Date	Delivered_Orders	Total_Revenue
0	4089	4089
1	839	839

Delivered-order totals:

No_Purchase_Date	Delivered_Orders	Total_Revenue
0	18853.00	24220170.09
1	2367.00	3038932.18

In [117...

```
# Identifying customers with no delivered orders

no_order_customers = clv_data[
    clv_data["Delivered_Orders"].isna()
]

print("Customers with no delivered orders:", len(no_order_customers))

print("\nCustomer activity status:")
print(no_order_customers["IsActive"].value_counts())

print("\nLoyalty tier:")
print(no_order_customers["LoyaltyTier"].value_counts())
```

Customers with no delivered orders: 72

Customer activity status:

```
IsActive
1    59
0    13
Name: count, dtype: int64
```

Loyalty tier:

```
LoyaltyTier
Bronze    34
Silver    24
Gold       9
Platinum   5
Name: count, dtype: int64
```

In [118...

```
# Handling missing purchase metrics

clv_data["Delivered_Orders"] = clv_data["Delivered_Orders"].fillna(0)
```

```

clv_data["Total_Revenue"] = clv_data["Total_Revenue"].fillna(0)
clv_data["Avg_Order_Value"] = clv_data["Avg_Order_Value"].fillna(0)

print(clv_data.isnull().sum())

```

```

CustomerID      0
LoyaltyTier     0
IsActive        0
Delivered_Orders 0
Total_Revenue   0
Avg_Order_Value 0
Last_Purchase_Date 911
Recency_Days    911
Tenure_Days     0
No_Purchase_Date 0
dtype: int64

```

In [119... *# Recency distribution for customers with valid purchase dates*

```

print(
    clv_data.loc[
        clv_data["Recency_Days"].notna(),
        "Recency_Days"
    ].describe()
)

```

```

count    4089.00
mean      120.74
std       97.56
min        0.00
25%       33.00
50%       95.00
75%      187.00
max      339.00
Name: Recency_Days, dtype: float64

```

In [120... *# Preparing recency for modeling*

```

median_recency = clv_data["Recency_Days"].median()

clv_data["Recency_Model"] = clv_data["Recency_Days"].fillna(
    median_recency
)

print("Median Recency Used:", median_recency)

print("\nMissing Recency_Model:")
print(clv_data["Recency_Model"].isna().sum())

print("\nCLV feature sample:")
print(
    clv_data[
        [
            "CustomerID",
            "Delivered_Orders",
            "Total_Revenue",
            "Avg_Order_Value",
            "Recency_Days",
            "Recency_Model",
            "Tenure_Days",
            "No_Purchase_Date"
        ]
    ].head()
)

```

Median Recency Used: 95.0

Missing Recency_Model:
0

CLV feature sample:

	CustomerID	Delivered_Orders	Total_Revenue	Avg_Order_Value	Recency_Days	\
0	CUST00001	5.00	5824.55	1164.91	94.00	
1	CUST00002	9.00	12822.93	1424.77	0.00	
2	CUST00003	4.00	4615.06	1153.77	NaN	
3	CUST00004	8.00	9809.07	1226.13	30.00	
4	CUST00005	5.00	7248.96	1449.79	157.00	

	Recency_Model	Tenure_Days	No_Purchase_Date
0	94.00	545	0
1	0.00	316	0
2	95.00	752	1
3	30.00	246	0
4	157.00	723	0

In [121... *# Creating behavior-based CLV target*

```
clv_data["Purchase_Frequency"] = (  
    clv_data["Delivered_Orders"] /  
    (clv_data["Tenure_Days"] / 365)  
)  
  
clv_data["CLV_Target"] = (  
    clv_data["Avg_Order_Value"] *  
    clv_data["Purchase_Frequency"] *  
    (clv_data["Tenure_Days"] / 365)  
)  
  
print(  
    clv_data[  
        [  
            "CustomerID",  
            "Delivered_Orders",  
            "Avg_Order_Value",  
            "Tenure_Days",  
            "Purchase_Frequency",  
            "CLV_Target"  
        ]  
    ].head()  
)
```

	CustomerID	Delivered_Orders	Avg_Order_Value	Tenure_Days	\
0	CUST00001	5.00	1164.91	545	
1	CUST00002	9.00	1424.77	316	
2	CUST00003	4.00	1153.77	752	
3	CUST00004	8.00	1226.13	246	
4	CUST00005	5.00	1449.79	723	

	Purchase_Frequency	CLV_Target
0	3.35	5824.55
1	10.40	12822.93
2	1.94	4615.06
3	11.87	9809.07
4	2.52	7248.96

In [122... *# Checking the delivered-order timeline*

```
delivered_orders = df5[  
    (df5["OrderStatus"] == "Delivered") &
```

```

(df5["OrderDateTime"].notna()
).copy()

print("Earliest Delivered Order:")
print(delivered_orders["OrderDateTime"].min())

print("\nLatest Delivered Order:")
print(delivered_orders["OrderDateTime"].max())

print("\nTotal Delivered Orders:")
print(len(delivered_orders))

```

Earliest Delivered Order:
2026-01-02 00:01:00

Latest Delivered Order:
2026-12-07 23:52:00

Total Delivered Orders:
8477

In [123...

```

# Validating CLV observation and prediction periods

cutoff_date = pd.Timestamp("2026-09-07")

observation_orders = delivered_orders[
    delivered_orders["OrderDateTime"] <= cutoff_date
]

future_orders = delivered_orders[
    delivered_orders["OrderDateTime"] > cutoff_date
]

print("Cutoff Date:", cutoff_date)

print("\nObservation Period Orders:")
print(len(observation_orders))

print("\nFuture Period Orders:")
print(len(future_orders))

print("\nObservation Revenue:")
print(round(observation_orders["FinalAmount"].sum(), 2))

print("\nFuture Revenue:")
print(round(future_orders["FinalAmount"].sum(), 2))

```

Cutoff Date: 2026-09-07 00:00:00

Observation Period Orders:
6254

Future Period Orders:
2223

Observation Revenue:
8035187.58

Future Revenue:
2864409.67

In [124...

```

# Creating future CLV target

future_clv = (

```

```

future_orders
.groupby("CustomerID")
.agg(
    Future_Orders=("OrderID", "nunique"),
    Future_Revenue=("FinalAmount", "sum")
)
.reset_index()
)

clv_data = clv_data.merge(
    future_clv,
    on="CustomerID",
    how="left"
)

# Customers with no future purchase have zero future CLV
clv_data["Future_Orders"] = clv_data["Future_Orders"].fillna(0)
clv_data["Future_Revenue"] = clv_data["Future_Revenue"].fillna(0)

print(
    clv_data[
        [
            "CustomerID",
            "Future_Orders",
            "Future_Revenue"
        ]
    ].head(10)
)

print("\nFuture CLV Summary:")
print(clv_data["Future_Revenue"].describe())

```

	CustomerID	Future_Orders	Future_Revenue
0	CUST00001	0.00	0.00
1	CUST00002	2.00	2105.20
2	CUST00003	0.00	0.00
3	CUST00004	2.00	2670.68
4	CUST00005	0.00	0.00
5	CUST00006	0.00	0.00
6	CUST00007	0.00	0.00
7	CUST00008	0.00	0.00
8	CUST00009	0.00	0.00
9	CUST00010	1.00	1876.59

Future CLV Summary:

```

count    5000.00
mean      572.88
std       964.48
min        0.00
25%        0.00
50%        0.00
75%     1017.51
max       7030.88

```

Name: Future_Revenue, dtype: float64

In [125... *# Validating future CLV target*

```

future_clv_summary = {
    "Total Customers": len(clv_data),
    "Customers With Future Purchase": (
        (clv_data["Future_Revenue"] > 0).sum()
    ),
    "Customers With No Future Purchase": (
        (clv_data["Future_Revenue"] == 0).sum()
    )
}

```

```

    ),
    "Future Purchase Rate (%)": (
        (clv_data["Future_Revenue"] > 0).mean() * 100
    )
}

for key, value in future_clv_summary.items():
    print(f"{key}: {value:.2f}" if isinstance(value, float) else f"{key}: {value}")

```

Total Customers: 5000

Customers With Future Purchase: 1797

Customers With No Future Purchase: 3203

Future Purchase Rate (%): 35.94

In [126... *# Building final CLV predictive feature table*

```

clv_features = clv_data[
    [
        "CustomerID",
        "LoyaltyTier",
        "IsActive",
        "Delivered_Orders",
        "Total_Revenue",
        "Avg_Order_Value",
        "Recency_Model",
        "Tenure_Days",
        "No_Purchase_Date",
        "Future_Revenue"
    ]
].copy()

print("Shape:", clv_features.shape)

print("\nColumns:")
print(clv_features.columns.tolist())

print("\nSample:")
print(clv_features.head())

```

Shape: (5000, 10)

Columns:

['CustomerID', 'LoyaltyTier', 'IsActive', 'Delivered_Orders', 'Total_Revenue', 'Avg_Order_Value', 'Recency_Model', 'Tenure_Days', 'No_Purchase_Date', 'Future_Revenue']

Sample:

	CustomerID	LoyaltyTier	IsActive	Delivered_Orders	Total_Revenue	\
0	CUST00001	Bronze	1	5.00	5824.55	
1	CUST00002	Bronze	1	9.00	12822.93	
2	CUST00003	Gold	0	4.00	4615.06	
3	CUST00004	Gold	1	8.00	9809.07	
4	CUST00005	Bronze	1	5.00	7248.96	

	Avg_Order_Value	Recency_Model	Tenure_Days	No_Purchase_Date	\
0	1164.91	94.00	545	0	
1	1424.77	0.00	316	0	
2	1153.77	95.00	752	1	
3	1226.13	30.00	246	0	
4	1449.79	157.00	723	0	

	Future_Revenue
0	0.00
1	2105.20
2	0.00
3	2670.68
4	0.00

In [127...

```
# Validating CLV modeling features

print("Data Types:")
print(clv_features.dtypes)

print("\nMissing Values:")
print(clv_features.isna().sum())

print("\nDuplicate Customers:")
print(clv_features["CustomerID"].duplicated().sum())

print("\nLoyalty Tier Distribution:")
print(clv_features["LoyaltyTier"].value_counts())

print("\nIsActive Distribution:")
print(clv_features["IsActive"].value_counts())
```

```
Data Types:
CustomerID      object
LoyaltyTier     object
IsActive        int64
Delivered_Orders float64
Total_Revenue   float64
Avg_Order_Value float64
Recency_Model   float64
Tenure_Days     int64
No_Purchase_Date int64
Future_Revenue  float64
dtype: object
```

```
Missing Values:
CustomerID      0
LoyaltyTier     0
IsActive        0
Delivered_Orders 0
Total_Revenue   0
Avg_Order_Value 0
Recency_Model   0
Tenure_Days     0
No_Purchase_Date 0
Future_Revenue  0
dtype: int64
```

```
Duplicate Customers:
0
```

```
Loyalty Tier Distribution:
LoyaltyTier
Bronze      2496
Silver      1536
Gold         750
Platinum     218
Name: count, dtype: int64
```

```
IsActive Distribution:
IsActive
1      4308
0       692
Name: count, dtype: int64
```

```
In [128... # Preparing features and target

X = clv_features[
    [
        "LoyaltyTier",
        "IsActive",
        "Delivered_Orders",
        "Total_Revenue",
        "Avg_Order_Value",
        "Recency_Model",
        "Tenure_Days",
        "No_Purchase_Date"
    ]
].copy()

y = clv_features["Future_Revenue"].copy()

# One-hot encode LoyaltyTier
X = pd.get_dummies(
    X,
```

```

columns=["LoyaltyTier"],
drop_first=True
)

print("X Shape:", X.shape)
print("y Shape:", y.shape)

print("\nX Columns:")
print(X.columns.tolist())

print("\nTarget Summary:")
print(y.describe())

```

X Shape: (5000, 10)

y Shape: (5000,)

X Columns:

['IsActive', 'Delivered_Orders', 'Total_Revenue', 'Avg_Order_Value', 'Recency_Model', 'Tenure_Days', 'No_Purchase_Date', 'LoyaltyTier_Gold', 'LoyaltyTier_Platinum', 'LoyaltyTier_Silver']

Target Summary:

```

count    5000.00
mean      572.88
std       964.48
min         0.00
25%        0.00
50%        0.00
75%       1017.51
max       7030.88

```

Name: Future_Revenue, dtype: float64

In [129...

```

from sklearn.model_selection import train_test_split

# Creating a binary indicator for stratification
y_purchase_flag = (y > 0).astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y_purchase_flag
)

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)

print("\nTraining future-purchase rate:")
print(round((y_train > 0).mean() * 100, 2), "%")

print("Test future-purchase rate:")
print(round((y_test > 0).mean() * 100, 2), "%")

```

X_train shape: (4000, 10)

X_test shape: (1000, 10)

y_train shape: (4000,)

y_test shape: (1000,)

Training future-purchase rate:

35.95 %

Test future-purchase rate:

35.9 %

```
In [130... # Training Random Forest CLV model

rf_clv = RandomForestRegressor(
    n_estimators=300,
    max_depth=10,
    min_samples_leaf=5,
    random_state=42,
    n_jobs=-1
)

rf_clv.fit(X_train, y_train)

print("Random Forest CLV model trained successfully.")
```

Random Forest CLV model trained successfully.

```
In [131... # Generating predictions

y_pred = rf_clv.predict(X_test)

# Evaluation metrics
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print("CLV Model Performance")
print("-" * 30)
print(f"MAE : ₹{mae:.2f}")
print(f"RMSE : ₹{rmse:.2f}")
print(f"R² : {r2:.4f}")
```

CLV Model Performance

```
-----
MAE : ₹234.86
RMSE : ₹543.61
R² : 0.6986
```

```
In [132... # Evaluating predictions for zero vs non-zero future revenue

evaluation_df = pd.DataFrame({
    "Actual_CLV": y_test.values,
    "Predicted_CLV": y_pred
})

evaluation_df["Actual_Purchase"] = (
    evaluation_df["Actual_CLV"] > 0
)

evaluation_df["Predicted_Purchase"] = (
    evaluation_df["Predicted_CLV"] > 0
)

print("Actual future purchasers:")
print(evaluation_df["Actual_Purchase"].sum())

print("\nActual non-purchasers:")
print((~evaluation_df["Actual_Purchase"]).sum())

print("\nAverage prediction for actual non-purchasers:")
print(
    round(
        evaluation_df.loc[
            ~evaluation_df["Actual_Purchase"],
            "Predicted_CLV"
        ].mean()
    )
)
```

```

        ].mean(),
        2
    )
)

print("\nAverage prediction for actual purchasers:")
print(
    round(
        evaluation_df.loc[
            evaluation_df["Actual_Purchase"],
            "Predicted_CLV"
        ].mean(),
        2
    )
)

```

Actual future purchasers:
359

Actual non-purchasers:
641

Average prediction for actual non-purchasers:
0.0

Average prediction for actual purchasers:
1631.67

In [133...

```

# Analyzing CLV prediction errors

evaluation_df["Prediction_Error"] = (
    evaluation_df["Predicted_CLV"] -
    evaluation_df["Actual_CLV"]
)

evaluation_df["Absolute_Error"] = (
    evaluation_df["Prediction_Error"].abs()
)

print("Error for non-purchasers:")
print(
    evaluation_df.loc[
        evaluation_df["Actual_CLV"] == 0,
        "Absolute_Error"
    ].describe()
)

print("\nError for purchasers:")
print(
    evaluation_df.loc[
        evaluation_df["Actual_CLV"] > 0,
        "Absolute_Error"
    ].describe()
)

```

Error for non-purchasers:

```
count    641.00
mean      0.00
std       0.00
min       0.00
25%      0.00
50%      0.00
75%      0.00
max       0.00
```

Name: Absolute_Error, dtype: float64

Error for purchasers:

```
count    359.00
mean    654.20
std     629.51
min      0.25
25%     223.60
50%     513.17
75%     874.76
max    4780.32
```

Name: Absolute_Error, dtype: float64

In [134...

```
# Feature importance

feature_importance = pd.DataFrame({
    "Feature": X_train.columns,
    "Importance": rf_clv.feature_importances_
}).sort_values(
    "Importance",
    ascending=False
).reset_index(drop=True)

print(feature_importance)
```

	Feature	Importance
0	Recency_Model	0.78
1	Avg_Order_Value	0.12
2	Total_Revenue	0.05
3	Tenure_Days	0.04
4	Delivered_Orders	0.00
5	LoyaltyTier_Silver	0.00
6	IsActive	0.00
7	LoyaltyTier_Gold	0.00
8	LoyaltyTier_Platinum	0.00
9	No_Purchase_Date	0.00

In [135...

```
# Actual vs predicted CLV

prediction_sample = pd.DataFrame({
    "Actual_CLV": y_test.values,
    "Predicted_CLV": y_pred
})

prediction_sample["Absolute_Error"] = (
    prediction_sample["Actual_CLV"] -
    prediction_sample["Predicted_CLV"]
).abs()

print(
    prediction_sample
    .sort_values("Actual_CLV", ascending=False)
    .head(10)
)
```

	Actual_CLV	Predicted_CLV	Absolute_Error
931	7030.88	2250.56	4780.32
199	5329.94	1789.67	3540.27
607	4953.98	1350.92	3603.06
968	4935.56	1733.65	3201.91
695	4800.88	3076.74	1724.14
13	4678.90	2027.79	2651.11
56	4580.41	1566.61	3013.80
322	4389.16	2295.63	2093.53
393	4352.35	2211.66	2140.69
351	4342.96	2256.96	2086.00

Customer Lifetime Value (CLV) Prediction

Objective

The objective of this analysis was to predict **future customer value** based on historical purchasing behavior and customer characteristics.

Instead of using the existing `TotalLifetimeValue` field directly, a future-revenue approach was used to create a genuine predictive CLV target.

Methodology

1. Customer Purchase Behavior

Historical delivered orders were aggregated at the customer level to calculate:

- Delivered Orders
- Total Revenue
- Average Order Value

Only orders with `OrderStatus = "Delivered"` were considered.

2. Recency

Customer recency was calculated as the number of days between the customer's latest delivered order and the reference date.

Reference date: December 7, 2026

The valid recency distribution was:

- Customers with valid purchase dates: **4,089**
- Median recency: **95 days**
- Mean recency: **120.74 days**
- Maximum recency: **339 days**

Customers without valid purchase timestamps were retained rather than having their recency artificially set to zero.

3. Customer Tenure

Customer tenure was calculated from the registration date to the reference date.

- Average tenure: **507.62 days**
- Median tenure: **510 days**
- Maximum tenure: **872 days**

4. Future CLV Target

A cutoff date of **September 7, 2026** was selected.

The dataset was divided into:

- **Observation period:** January 2 – September 7, 2026
- **Future prediction period:** September 8 – December 7, 2026

The future CLV target was defined as:

Future CLV = Customer revenue generated during the future prediction period

This approach avoids using the pre-existing lifetime-value field as the prediction target and prevents historical revenue from being directly reused as the target.

5. Target Validation

The future-period analysis contained:

- **5,000 customers**
- **1,797 customers with future purchases**
- **3,203 customers with no future purchases**
- Future purchase rate: **35.94%**

Future revenue statistics:

- Mean: **₹572.88**
- Median: **₹0**
- 75th percentile: **₹1,017.51**
- Maximum: **₹7,030.88**

The zero median reflects the fact that most customers did not generate revenue during the three-month prediction period.

Predictive Features

The model used the following customer-level features:

- `IsActive`
- `Delivered_Orders`
- `Total_Revenue`
- `Avg_Order_Value`
- `Recency_Model`
- `Tenure_Days`
- `No_Purchase_Date`

- `LoyaltyTier`

`CustomerID` was excluded because it is only an identifier.

The existing `TotalLifetimeValue` field was also excluded to avoid relying on a precomputed CLV-like metric.

Model

A **Random Forest Regressor** was selected because it can capture nonlinear relationships and interactions between customer behavioral features without requiring feature scaling.

Model configuration:

- 300 trees
- Maximum depth: 10
- Minimum samples per leaf: 5
- Random state: 42

The dataset was split into:

- **80% training:** 4,000 customers
- **20% testing:** 1,000 customers

Stratification was performed using the future-purchase indicator to maintain a similar purchaser/non-purchaser distribution across the training and test sets.

Model Performance

Metric	Result
MAE	₹234.86
RMSE	₹543.61
R ²	0.6986

The model explains approximately **69.86% of the variation in future customer revenue** on the test set.

The average absolute prediction error was approximately **₹235**.

Customer Purchase Behavior Validation

Within the test set:

- **359 customers** generated future revenue.
- **641 customers** generated no future revenue.

The model predicted an average future CLV of:

- **₹0** for actual non-purchasers

- ₹1,631.67 for actual purchasers

This indicates that the model effectively separates customers with no future revenue from customers who generate future revenue.

However, prediction accuracy decreases for high-value customers. For example:

Actual Future CLV	Predicted CLV	Absolute Error
₹7,030.88	₹2,250.56	₹4,780.32
₹5,329.94	₹1,789.67	₹3,540.27
₹4,953.98	₹1,350.92	₹3,603.06
₹4,935.56	₹1,733.65	₹3,201.91

This shows that the model tends to **underpredict exceptionally high-value customers**.

Feature Importance

Feature	Importance
Recency	78%
Average Order Value	12%
Total Revenue	5%
Tenure	4%
Delivered Orders	~0%
Loyalty Tier	~0%
IsActive	~0%
No Purchase Date	~0%

Key Finding

Recency was by far the strongest predictive feature, accounting for approximately **78% of the model's feature importance**.

Average Order Value was the second-largest contributor at approximately **12%**, followed by historical revenue and customer tenure.

This indicates that recent purchasing behavior was highly informative when predicting future customer value within this dataset.

Feature importance represents the model's predictive behavior and should not be interpreted as proof of causation.

Business Insights

1. **Recent customer activity is highly valuable for CLV prediction.** Customers with more recent purchases were more strongly associated with future revenue generation.
 2. **Average order value provides additional predictive power.** Customers with larger historical baskets tended to have higher predicted future value.
 3. **The model successfully identifies zero-value customers.** It predicted ₹0 for the customers in the test set who generated no future revenue.
 4. **High-value customers remain difficult to estimate precisely.** The model underpredicted several customers with exceptionally high future revenue.
 5. **Customer segmentation can support targeted retention strategies.** Recency and historical spending behavior can be used to identify customers with stronger potential future value.
-

Conclusion

The CLV analysis developed a **behavior-based future revenue prediction model** rather than relying on the dataset's existing lifetime-value field.

The Random Forest model achieved an **R² of 0.6986**, with **₹234.86 MAE** and **₹543.61 RMSE**.

The strongest model driver was **customer recency**, highlighting the importance of recent purchasing behavior in predicting future customer value.

The model provides a useful foundation for customer segmentation, retention analysis, and prioritization of high-potential customers, while recognizing that exceptionally high-value customers are harder to predict accurately.

Stock Out Prediction

In [136...

```
# Inventory Data Overview

print("Shape:", df8.shape)

print("\nColumns:")
print(df8.columns.tolist())

print("\nStock Status:")
print(df8["StockStatus"].value_counts())

print("\nMissing Values:")
print(df8.isna().sum())
```

Shape: (25000, 9)

Columns:

```
['InventoryID', 'StoreID', 'ProductID', 'StockLevel', 'ReorderLevel', 'StockStatus', 'LastRestockedDate', 'CapacityPercent', 'LastUpdated']
```

Stock Status:

StockStatus

In Stock 18760

Low Stock 3764

Out of Stock 2476

Name: count, dtype: int64

Missing Values:

InventoryID 0

StoreID 0

ProductID 0

StockLevel 0

ReorderLevel 0

StockStatus 0

LastRestockedDate 0

CapacityPercent 0

LastUpdated 25000

dtype: int64

```
In [137... # Creating stock-out target

df8["Stock_Out"] = (
    df8["StockStatus"] == "Out of Stock"
).astype(int)

print("Stock-Out Target Distribution:")
print(df8["Stock_Out"].value_counts())

print("\nStock-Out Rate:")
print(round(df8["Stock_Out"].mean() * 100, 2), "%")
```

Stock-Out Target Distribution:

Stock_Out

0 22524

1 2476

Name: count, dtype: int64

Stock-Out Rate:

9.9 %

```
In [138... # Feature Engineering

df8["Stock_Gap"] = (
    df8["StockLevel"] - df8["ReorderLevel"]
)

df8["Stock_to_Reorder_Ratio"] = (
    df8["StockLevel"] /
    df8["ReorderLevel"].replace(0, np.nan)
)

df8["Low_Stock_Flag"] = (
    df8["StockLevel"] <= df8["ReorderLevel"]
).astype(int)

print(
    df8[
        [
```

```

        "StockLevel",
        "ReorderLevel",
        "Stock_Gap",
        "Stock_to_Reorder_Ratio",
        "Low_Stock_Flag",
        "CapacityPercent",
        "Stock_Out"
    ]
    ].head()
)

```

	StockLevel	ReorderLevel	Stock_Gap	Stock_to_Reorder_Ratio	\
0	412	48	364	8.58	
1	47	41	6	1.15	
2	79	29	50	2.72	
3	89	24	65	3.71	
4	73	23	50	3.17	

	Low_Stock_Flag	CapacityPercent	Stock_Out
0	0	6.74	0
1	0	6.18	0
2	0	9.39	0
3	0	13.98	0
4	0	14.64	0

In [139... *# Preparing modeling dataset*

```

X = df8[
    [
        "StockLevel",
        "ReorderLevel",
        "Stock_Gap",
        "Stock_to_Reorder_Ratio",
        "Low_Stock_Flag",
        "CapacityPercent"
    ]
].copy()

y = df8["Stock_Out"].copy()

# Handle any infinite or missing ratio values
X = X.replace([np.inf, -np.inf], np.nan)
X["Stock_to_Reorder_Ratio"] = X["Stock_to_Reorder_Ratio"].fillna(0)

print("X Shape:", X.shape)
print("y Shape:", y.shape)

print("\nMissing Values:")
print(X.isna().sum())

```

```

X Shape: (25000, 6)
y Shape: (25000,)

```

```

Missing Values:
StockLevel          0
ReorderLevel        0
Stock_Gap           0
Stock_to_Reorder_Ratio  0
Low_Stock_Flag      0
CapacityPercent     0
dtype: int64

```

In [140... *# Train/Test Split*

```

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("X_train:", X_train.shape)
print("X_test:", X_test.shape)

print("\nTraining Stock-Out Rate:")
print(round(y_train.mean() * 100, 2), "%")

print("Test Stock-Out Rate:")
print(round(y_test.mean() * 100, 2), "%")

```

X_train: (20000, 6)

X_test: (5000, 6)

Training Stock-Out Rate:

9.9 %

Test Stock-Out Rate:

9.9 %

In [141...

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score
)

# Training model

rf_stockout = RandomForestClassifier(
    n_estimators=300,
    max_depth=10,
    min_samples_leaf=5,
    random_state=42,
    n_jobs=-1,
    class_weight="balanced"
)

rf_stockout.fit(X_train, y_train)

# Predictions
y_pred = rf_stockout.predict(X_test)
y_prob = rf_stockout.predict_proba(X_test)[:, 1]

print("Stock-Out Prediction Performance")
print("-" * 35)
print(f"Accuracy : {accuracy_score(y_test, y_pred):.4f}")
print(f"Precision: {precision_score(y_test, y_pred):.4f}")
print(f"Recall    : {recall_score(y_test, y_pred):.4f}")
print(f"F1 Score  : {f1_score(y_test, y_pred):.4f}")
print(f"ROC-AUC   : {roc_auc_score(y_test, y_prob):.4f}")

```

Stock-Out Prediction Performance

Accuracy : 1.0000
Precision: 1.0000
Recall : 1.0000
F1 Score : 1.0000
ROC-AUC : 1.0000

In [142...

```
# Rebuilding features without leakage-prone variables

X = df8[
    [
        "ReorderLevel",
        "CapacityPercent"
    ]
].copy()

y = df8["Stock_Out"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

rf_stockout = RandomForestClassifier(
    n_estimators=300,
    max_depth=10,
    min_samples_leaf=5,
    random_state=42,
    n_jobs=-1,
    class_weight="balanced"
)

rf_stockout.fit(X_train, y_train)

y_pred = rf_stockout.predict(X_test)
y_prob = rf_stockout.predict_proba(X_test)[:, 1]

print("Accuracy :", round(accuracy_score(y_test, y_pred), 4))
print("Precision:", round(precision_score(y_test, y_pred), 4))
print("Recall   :", round(recall_score(y_test, y_pred), 4))
print("F1 Score :", round(f1_score(y_test, y_pred), 4))
print("ROC-AUC  :", round(roc_auc_score(y_test, y_prob), 4))
```

Accuracy : 0.736
Precision: 0.1053
Recall : 0.2222
F1 Score : 0.1429
ROC-AUC : 0.5193

In [143...

```
# Feature importance

feature_importance = pd.DataFrame({
    "Feature": X_train.columns,
    "Importance": rf_stockout.feature_importances_
}).sort_values(
    "Importance",
    ascending=False
)

print(feature_importance)
```

	Feature	Importance
1	CapacityPercent	0.79
0	ReorderLevel	0.21

Stock-Out Prediction Analysis

Objective

The objective was to predict whether an inventory record would be **Out of Stock** using available inventory-level information.

1. Inventory Overview

The inventory dataset contained:

- **25,000 inventory records**
- **18,760** In Stock
- **3,764** Low Stock
- **2,476** Out of Stock

The overall stock-out rate was **9.9%**.

The `LastUpdated` field was completely missing and was excluded from the analysis.

2. Target Definition

A binary target variable, `Stock_Out`, was created:

- `1` → Out of Stock
- `0` → In Stock or Low Stock

Target distribution:

- **22,524 non-stock-out records**
 - **2,476 stock-out records**
-

3. Feature Engineering

Initial inventory signals included:

- Stock Level
- Reorder Level
- Stock Gap
- Stock-to-Reorder Ratio
- Low Stock Flag
- Capacity Percent

However, features directly derived from current stock status were identified as **target-leakage risks** and were removed from the final predictive model.

The final model used:

- ReorderLevel
- CapacityPercent

4. Model

A **Random Forest Classifier** was used.

The dataset was divided into:

- **80% training:** 20,000 records
- **20% testing:** 5,000 records

Stratification maintained the **9.9% stock-out rate** in both datasets.

5. Model Performance

Metric	Result
Accuracy	73.60%
Precision	10.53%
Recall	22.22%
F1 Score	14.29%
ROC-AUC	0.5193

The initial model produced perfect 1.00 metrics, but investigation revealed that the model was benefiting from leakage-prone variables. After removing those variables, performance dropped substantially.

6. Feature Importance

Feature	Importance
CapacityPercent	79%

Feature	Importance
ReorderLevel	21%

Capacity percentage was the strongest feature among the non-leaking variables used in the final model.

7. Key Finding

The final ROC-AUC of **0.5193** indicates that the available non-leaking inventory features provide **very limited predictive power** for stock-outs.

This is an important analytical finding rather than a failed analysis. The results indicate that stronger predictive variables—such as historical demand, sales velocity, product-level consumption, seasonality, and replenishment history—would likely be required to build a more useful stock-out prediction system.

Conclusion

The analysis demonstrated the importance of **target-leakage detection** in predictive modeling. Rather than reporting artificially perfect model performance, leakage-prone variables were removed and the model was evaluated using more defensible features.

The analysis therefore provides a realistic baseline for future inventory stock-out prediction.

Demand Forecasting

In [144...

```
# Filtering delivered orders with valid order dates
demand_data = df5[
    (df5["OrderStatus"] == "Delivered") &
    (df5["OrderDateTime"].notna())
].copy()

# Extract date
demand_data["OrderDate"] = demand_data["OrderDateTime"].dt.date

# Daily demand
daily_demand = (
    demand_data
    .groupby("OrderDate")
    .agg(
        Demand=("OrderID", "nunique")
    )
    .reset_index()
)

# Convert date to datetime
daily_demand["OrderDate"] = pd.to_datetime(daily_demand["OrderDate"])

# Sort by date
daily_demand = daily_demand.sort_values("OrderDate").reset_index(drop=True)

daily_demand.head()
```

Out[144...

	OrderDate	Demand
0	2026-01-02	127
1	2026-01-03	100
2	2026-01-04	98
3	2026-01-05	117
4	2026-01-06	114

In [145...

```
# Creating time-based features
daily_demand["Day_of_Week"] = daily_demand["OrderDate"].dt.day_name()
daily_demand["Month"] = daily_demand["OrderDate"].dt.month_name()

# Average demand by day of week
weekly_demand = (
    daily_demand
    .groupby("Day_of_Week")["Demand"]
    .mean()
    .reindex([
        "Monday", "Tuesday", "Wednesday",
        "Thursday", "Friday", "Saturday", "Sunday"
    ])
)

print("Average Demand by Day of Week:")
print(weekly_demand)

# Average demand by month
monthly_demand = daily_demand.groupby(
    daily_demand["OrderDate"].dt.to_period("M")
)["Demand"].mean()

print("\nAverage Demand by Month:")
print(monthly_demand)
```

Average Demand by Day of Week:

Day_of_Week

Monday	118.18
Tuesday	114.80
Wednesday	117.60
Thursday	123.10
Friday	118.91
Saturday	116.18
Sunday	115.11

Name: Demand, dtype: float64

Average Demand by Month:

OrderDate

2026-01	111.17
2026-02	123.67
2026-03	124.67
2026-04	121.00
2026-05	108.67
2026-06	121.67
2026-07	111.33
2026-08	124.50
2026-09	114.67
2026-10	117.17
2026-11	115.67
2026-12	118.67

Freq: M, Name: Demand, dtype: float64

In [146...

```
# Time features
daily_demand["Day_Number"] = daily_demand["OrderDate"].dt.day
daily_demand["Day_of_Week_Num"] = daily_demand["OrderDate"].dt.dayofweek
daily_demand["Month_Num"] = daily_demand["OrderDate"].dt.month

# Lag features
daily_demand["Lag_1"] = daily_demand["Demand"].shift(1)
daily_demand["Lag_7"] = daily_demand["Demand"].shift(7)

# Rolling averages
daily_demand["Rolling_7_Day"] = (
    daily_demand["Demand"]
    .shift(1)
    .rolling(7)
    .mean()
)

daily_demand["Rolling_14_Day"] = (
    daily_demand["Demand"]
    .shift(1)
    .rolling(14)
    .mean()
)

daily_demand.head(15)
```

Out[146...

	OrderDate	Demand	Day_of_Week	Month	Day_Number	Day_of_Week_Num	Month_Num	Lag
0	2026-01-02	127	Friday	January	2	4	1	Na
1	2026-01-03	100	Saturday	January	3	5	1	127.0
2	2026-01-04	98	Sunday	January	4	6	1	100.0
3	2026-01-05	117	Monday	January	5	0	1	98.0
4	2026-01-06	114	Tuesday	January	6	1	1	117.0
5	2026-01-07	111	Wednesday	January	7	2	1	114.0
6	2026-02-02	126	Monday	February	2	0	2	111.0
7	2026-02-03	118	Tuesday	February	3	1	2	126.0
8	2026-02-04	128	Wednesday	February	4	2	2	118.0
9	2026-02-05	107	Thursday	February	5	3	2	128.0
10	2026-02-06	127	Friday	February	6	4	2	107.0
11	2026-02-07	136	Saturday	February	7	5	2	127.0
12	2026-03-02	136	Monday	March	2	0	3	136.0
13	2026-03-03	107	Tuesday	March	3	1	3	136.0
14	2026-03-04	137	Wednesday	March	4	2	3	107.0



In [147...

```
# Features for demand forecasting
features = [
    "Day_Number",
    "Day_of_Week_Num",
    "Month_Num",
    "Lag_1",
    "Lag_7",
    "Rolling_7_Day",
    "Rolling_14_Day"
]

target = "Demand"

# Remove rows with missing feature values
model_data = daily_demand.dropna(
    subset=features + [target]
).copy()
```

```

X = model_data[features]
y = model_data[target]

print("Modeling Dataset Shape:", model_data.shape)
print("Features:", features)
print("Target:", target)
print("\nMissing Values:")
print(X.isnull().sum())

```

Modeling Dataset Shape: (58, 11)

Features: ['Day_Number', 'Day_of_Week_Num', 'Month_Num', 'Lag_1', 'Lag_7', 'Rolling_7_Day', 'Rolling_14_Day']

Target: Demand

Missing Values:

```

Day_Number      0
Day_of_Week_Num 0
Month_Num        0
Lag_1            0
Lag_7            0
Rolling_7_Day    0
Rolling_14_Day   0
dtype: int64

```

In [148...

```

# Time-based train/test split
split_index = int(len(model_data) * 0.8)

train_data = model_data.iloc[:split_index]
test_data = model_data.iloc[split_index:]

X_train = train_data[features]
y_train = train_data[target]

X_test = test_data[features]
y_test = test_data[target]

print("Training Samples:", len(X_train))
print("Testing Samples:", len(X_test))

print("\nTraining Period:")
print(train_data["OrderDate"].min(), "to", train_data["OrderDate"].max())

print("\nTesting Period:")
print(test_data["OrderDate"].min(), "to", test_data["OrderDate"].max())

```

Training Samples: 46

Testing Samples: 12

Training Period:

2026-03-04 00:00:00 to 2026-10-07 00:00:00

Testing Period:

2026-11-02 00:00:00 to 2026-12-07 00:00:00

In [149...

```

# Train model
model = RandomForestRegressor(
    n_estimators=300,
    max_depth=10,
    min_samples_leaf=2,
    random_state=42
)

model.fit(X_train, y_train)

```

```

# Predictions
y_pred = model.predict(X_test)

# Evaluation
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print("Demand Forecasting Performance")
print("-----")
print(f"MAE : {mae:.2f}")
print(f"RMSE : {rmse:.2f}")
print(f"R² : {r2:.4f}")

```

Demand Forecasting Performance

MAE : 6.97
 RMSE : 8.94
 R² : -0.8052

In [150...

```

forecast_results = test_data[["OrderDate", "Demand"]].copy()

forecast_results["Predicted_Demand"] = np.round(y_pred, 0)

forecast_results["Forecast_Error"] = (
    forecast_results["Demand"] -
    forecast_results["Predicted_Demand"]
)

print(forecast_results)

```

	OrderDate	Demand	Predicted_Demand	Forecast_Error
60	2026-11-02	125	131.00	-6.00
61	2026-11-03	124	116.00	8.00
62	2026-11-04	104	117.00	-13.00
63	2026-11-05	111	116.00	-5.00
64	2026-11-06	112	114.00	-2.00
65	2026-11-07	118	115.00	3.00
66	2026-12-02	110	131.00	-21.00
67	2026-12-03	127	116.00	11.00
68	2026-12-04	118	114.00	4.00
69	2026-12-05	116	117.00	-1.00
70	2026-12-06	123	114.00	9.00
71	2026-12-07	118	117.00	1.00

In [151...

```

print("\nAverage Actual Demand:",
      forecast_results["Demand"].mean())

print("Average Predicted Demand:",
      forecast_results["Predicted_Demand"].mean())

```

Average Actual Demand: 117.16666666666667

Average Predicted Demand: 118.16666666666667

Demand Forecasting Analysis

Objective

Forecast daily order demand using historical delivered-order patterns and time-based features to support operational planning, inventory management, and delivery capacity decisions.

Step 1 — Demand Data Preparation

Used delivered orders from the orders dataset and aggregated them by date.

Demand Definition:

Daily number of unique delivered orders.

The resulting dataset contained daily demand observations from **January 2, 2026 to December 7, 2026**.

Example:

Order Date	Demand
2026-01-02	127
2026-01-03	100
2026-01-04	98
2026-01-05	117
2026-01-06	114

Step 2 — Demand Trend & Seasonality Analysis

Analyzed demand patterns by day of week and month.

Average Demand by Day

Day	Average Demand
Monday	118.18
Tuesday	114.80
Wednesday	117.60
Thursday	123.10
Friday	118.91
Saturday	116.18
Sunday	115.11

Thursday recorded the highest average daily demand at **123.10 orders**.

Average Demand by Month

Month	Average Demand
January	111.17
February	123.67
March	124.67
April	121.00

Month	Average Demand
May	108.67
June	121.67
July	111.33
August	124.50
September	114.67
October	117.17
November	115.67
December	118.67

March recorded the highest monthly average demand at **124.67 orders**, while May recorded the lowest at **108.67 orders**.

Step 3 — Feature Engineering

Created time-based and historical demand features:

- Day of month
- Day of week
- Month
- Lag 1 day
- Lag 7 days
- 7-day rolling average
- 14-day rolling average

Lag and rolling features were shifted before calculation to prevent using current-day demand when making predictions.

Step 4 — Modeling Dataset

After removing rows where lag and rolling features were unavailable:

- **Modeling observations:** 58
- **Features:** 7
- **Target:** Daily Demand
- **Missing feature values:** 0

Model Features

Day_Number
Day_of_Week_Num
Month_Num
Lag_1
Lag_7
Rolling_7_Day
Rolling_14_Day

Step 5 — Time-Based Train/Test Split

A chronological split was used instead of random sampling to preserve the forecasting structure.

- **Training observations:** 46
- **Testing observations:** 12

Training period: March 4, 2026 – October 7, 2026

Testing period: November 2, 2026 – December 7, 2026

Step 6 — Forecasting Model & Evaluation

A **Random Forest Regressor** was trained using:

```
n_estimators = 300
max_depth = 10
min_samples_leaf = 2
random_state = 42
```

Model Performance

Metric	Result
MAE	6.97 orders
RMSE	8.94 orders
R ²	-0.8052

The model's average absolute error was approximately **7 orders per day**.

The negative R² indicates that the model had limited ability to explain day-to-day demand variation in the unseen test period.

Step 7 — Forecast Results & Business Insights

Average Demand

- **Actual average demand:** 117.17 orders/day
- **Predicted average demand:** 118.17 orders/day
- **Difference:** approximately 1 order/day

The forecast was therefore close to the overall average demand level, although it was less effective at capturing daily fluctuations.

Forecast Examples

Date	Actual	Predicted	Error
Nov 2	125	131	-6
Nov 3	124	116	8
Nov 4	104	117	-13

Date	Actual	Predicted	Error
Nov 5	111	116	-5
Nov 6	112	114	-2
Dec 2	110	131	-21
Dec 3	127	116	11
Dec 7	118	117	1

Business Insights

1. Average demand remained around **117 orders per day**.
2. Thursday showed the highest average demand among the days of the week.
3. March and August showed relatively higher monthly demand.
4. The model provides reasonably close average-demand estimates but does not reliably capture short-term fluctuations.
5. A stronger forecasting system would benefit from additional historical observations and external demand drivers such as promotions, holidays, weather, city/store-level demand, and historical sales velocity.

Final Conclusion

The demand forecasting analysis established a baseline machine-learning forecasting approach using historical demand, lag features, rolling averages, and calendar variables. The model achieved an **MAE of 6.97 orders**, while the negative R^2 highlighted the limitations of the available historical and explanatory data.

Rather than relying on an artificially strong model score, the analysis identified the data limitations and provided clear directions for improving the forecasting framework.

In []: